

Practical Runtime Enforcement of First-Order Temporal Requirements

ANONYMOUS AUTHOR(S)

Runtime enforcers observe a system’s behavior and exert control over it to ensure that the system always adheres to its requirements. Many natural requirements not only formulate restrictions on the system’s present behavior, but also impose obligations on its future behavior; for instance, agentic security may require personal data collected during a run to be erased by some deadline. In an ever-growing compliance landscape, real-world systems may be subject to hundreds of such rules. However, existing enforcement mechanisms supporting complex policies are often too slow for production software; instead, developers may resort to non-temporal access control mechanisms and ad-hoc instrumentation, but these scale poorly with requirement complexity.

To address this problem, we introduce an efficient algorithm and tool for enforcing complex temporal requirements and demonstrate its performance as a rule-enforcement engine. Specifically, we identify a fragment of Metric First-Order Temporal Logic (MFOTL) that can be enforced with low runtime complexity while supporting a rich family of practically relevant requirements, including obligations. We then design a monitoring algorithm for this fragment and implement it in ENFLASH, a novel tool that enforces requirements with latency up to 44× lower than EnGuard, the previous state-of-the-art enforcer, on standard benchmarks, and up to three orders of magnitude lower on security policies for LLM agents. This performance is achieved by compiling requirements into imperative programs that we efficiently interpret. We evaluate ENFLASH both standalone, on existing benchmarks, and integrated in web applications as an enforcement backend. We demonstrate that it can enforce a substantial 400-line MFOTL specification of the GDPR in a social network while adding under 15 ms to each page view, which is sufficient for most interactive and real-time applications.

CCS Concepts: • **Security and privacy** → **Logic and verification**; • **Theory of computation** → *Modal and temporal logics*.

Additional Key Words and Phrases: runtime enforcement, security, privacy, logic

ACM Reference Format:

Anonymous Author(s). 2026. Practical Runtime Enforcement of First-Order Temporal Requirements. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE ’27)*. ACM, New York, NY, USA, 25 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Modern systems must comply with increasingly complex requirements and regulations. Rather than auditing these requirements ex post, in *runtime enforcement* [8, 48] an *enforcer* observes the system’s behavior and ensures compliance at runtime by suppressing or causing system actions.

Requirements can be broadly classified as *provisions*, whose satisfaction depends on past and present system behavior, and *obligations*, whose satisfaction depends on its future behavior [24]. An example provision is enforcing access to confidential data only when a prior approval was granted, whereas an obligation would be that any temporary copy made of confidential data must be deleted within a given deadline. Both of these requirements are *temporal* and *first-order* (i.e., they relate

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE ’27, Shenzhen, China

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2027/07

<https://doi.org/XXXXXXX.XXXXXXX>

data-carrying events, like file identifiers, over time). However, they require different enforcement capabilities: an enforcer can simply *react* to an attempted system action to enforce a provision, while it must *proactively* act to enforce an obligation. Runtime enforcement plays a crucial role in numerous domains such as AI agent security: agent behavior cannot be statically analyzed, but only observed, e.g., as a sequence of tool invocations. The example provision and obligation above readily apply in this context with an enforcer mediating tool invocations to ensure policy compliance and retaining responsibility for pending obligations independently of the agent’s subsequent behavior.

Recent industrial developments reinforce the relevance of temporal first-order requirements for agentic software and put forward enforcement performance as a key challenge for its practical adoption. AWS’s Cedar access-control (AC) mechanism [14] is a fast, non-temporal, first-order enforcer designed for a very limited set of provisions: those that depend only on the current system state without considering its past behavior. To alleviate this limitation, AWS recently proposed Dogwood [9], which combines Cedar with Metric First-Order Temporal Logic (MFOTL) [12] to express provisions. While MFOTL offers both past and future temporal operators to express both complex provisions and obligations, Dogwood only supports provisions, with obligations identified as future work.

Other enforcers based on MFOTL, like EnfGuard [28] and WhyEnf [27] support both provisions and obligations. Unfortunately, these tools’ latency is high and can grow with the size of the enforcer’s state. This leads to latencies of the order of seconds even for relatively simple requirements, making these tools unsuitable to use interactive and real-time applications.

An alternative to out-of-box enforcers is to directly implement ad-hoc, specialized ones to achieve good performance. However, this is brittle and scales poorly. Abandoning declarative specifications of provisions and obligations, underpinned with formal semantics, introduces a new problem: justifying that the ad-hoc implementation behaves as intended requires reasoning about the much more complex and often unavailable formal semantics of the implementation language. This, combined with the fact that in practice one must implement hundreds of interacting requirements and encode their temporal dependencies by hand, makes this error-prone and hard to evolve [39].

To improve upon the status quo, software developers need a tool that combines three properties: declarative specifications of data-dependent provisions and obligations, sound generation of reactive and proactive enforcement, and runtime performance efficient enough for interactive applications, which require response times on the order of fractions of a second.

In this paper, we introduce ENFLASH, a novel tool for enforcing first-order temporal provisions and obligations with low, predictable latency. Unlike previous tools that *interpret* MFOTL formulae [26–28], ENFLASH *compiles* them into interpreted imperative programs each describing an efficient enforcer for the corresponding formula, and then executes them. The generated programs are provably correct and their execution complexity can be statically bounded. The MFOTL fragment supported by ENFLASH is large enough to express a large collection of provisions and obligations that arise in practice, including requirements involving aggregations and user-defined operators. Additionally, ENFLASH lifts the key limitation of previous enforcers [26–28] that could not simultaneously cause and suppress the same event with different parameters; its compiler optimizes the order of rule evaluation and fixpoint computations to reduce overhead while preserving soundness.

We evaluate ENFLASH’s expressivity and performance using existing benchmarks [14, 28] and deploy it as an enforcement backend for two mid-sized web applications: a social network platform instrumented for compliance with a 400-line specification of the General Data Protection Regulation (GDPR), and an event management platform accompanied by a broad range of access-control rules. We also enforce security policies on the tool calls of an LLM banking agent under prompt-injection attacks. Compared with EnfGuard [28], the previous state-of-the-art MFOTL enforcer, ENFLASH’s latency is 3–44× lower on 17 of the 20 benchmark formulae both process (EnfGuard times out

on 19 others) and 20–5,570× lower on the agent’s policies; in web applications, enforcement adds 1–15 ms per page view, compatible with production use.

After reviewing enforcement preliminaries (Section 2), we make the following contributions:

- We design an imperative DSL called EF that captures the behavior of proactive first-order enforcers, give it a formal semantics, and bound EF’s evaluation time complexity (Section 3).
- We introduce a novel compilation procedure from MFOTL to EF, generating sound enforcers for an expressive fragment of MFOTL (Section 4).
- We implement our algorithms in ENFLASH, and evaluate it both on existing enforcement benchmarks and as an instrumentation backend for web applications and controlling LLM agents. We show that ENFLASH enforces specifications with hundreds of rules with modest latency, with 3–44× lower latency than EnfGuard on most benchmark formulae (Section 5).

Overall, ENFLASH is the first enforcer for first-order temporal provisions and obligations offering practical performance (Section 6). With the exception of the complexity results in Section 3.3, all our theorems are formally verified in Lean 4 [37] (marked with $\sqsubseteq \mathbb{N}$) for the core logic presented in this paper.

2 Runtime Enforcement

We introduce traces that model system executions, metric-first order temporal logic (MFOTL), and enforcers.

2.1 Traces

Let $\mathbb{E}$ be a finite set of *event names* with arity function $\iota : \mathbb{E} \rightarrow \mathbb{N}$. An *event* is a pair $(e, \vec{d}) \in \mathbb{E} \times \mathbb{D}^{\iota(e)}$ of an event name e and its $\iota(e)$ arguments, tuple $\vec{d}$. A *signature* $\Sigma = (\mathbb{D}, \mathbb{E}, \iota)$ defines the set of *databases* $\mathbb{DB} := \mathcal{P}(\{(e, \vec{d}) \mid e \in \mathbb{E}, \vec{d} \in \mathbb{D}^{\iota(e)}\})$. For $E \subseteq \mathbb{E}$, let $\mathbb{DB}_E := \{D \in \mathbb{DB} \mid \forall (e, \vec{d}) \in D. e \in E\}$ denote databases with event names in E .

Example 2.1. Consider a file management system. Files can be accessed (‘used’) or deleted, and users can request deletion of their data, give consent for files of some category to be used, or revoke said consent. We model this with the signature $\Sigma_0 = (\mathbb{N}, \mathbb{E}_0, \iota_0)$, where $\mathbb{E}_0 = \{\text{use}, \text{consent}, \text{revoke}, \text{delete}, \text{deletion_request}\}$, $\iota_0(\text{use}) = 3$, and $\iota_0(\text{delete}) = \iota_0(\text{deletion_request}) = \iota_0(\text{consent}) = \iota_0(\text{revoke}) = 2$. The event $\text{use}(c, d, u)$ denotes ‘system uses user u ’s data d from category c ’, $\text{delete}(c, d)$ denotes ‘user u ’s data d is deleted’, $\text{deletion_request}(d, u)$ denotes ‘user u requests deletion of data d ’, $\text{consent}(u, c)$ denotes ‘user u provides consent for category c ’, and $\text{revoke}(u, c)$ denotes ‘user u revokes consent for category c ’.

A *trace* σ is a sequence $\langle (\tau_i, D_i) \rangle_{0 \leq i < k}$ of length $|\sigma| = k \in \mathbb{N} \cup \{\infty\}$ of pairs of timestamps $\tau_i \in \mathbb{N}$ and databases $D_i \in \mathbb{DB}$, where timestamps grow *monotonically* ($\tau_i \leq \tau_{i+1}$ for all i with $i + 1 < |\sigma|$) and *progress* (if $|\sigma| = \infty$, then $\forall \tau. \exists i. \tau < \tau_i$). An index $0 \leq i < |\sigma|$ in a trace σ is called a *time-point*. The empty trace is denoted by ε , the set of all traces by $\mathbb{T}$.

Example 2.2. Consider two traces of the above system:

$$\begin{aligned} \sigma_1 &= (10, \{\text{consent}(1, 1)\}), (50, \{\text{use}(1, 3, 1), \text{use}(2, 1, 1)\}), \dots \\ \sigma_2 &= (10, \{\text{deletion_request}(1, 1)\}), (50, \{\text{use}(1, 3, 1)\}), \dots \end{aligned}$$

In σ_1 , user 1 provides consent for category 1 at time-point 0 with timestamp 10; at time-point 1 with timestamp 50, the system uses user 1’s data 3 (with category 1) and user 1’s data 1 (with category 2). In σ_2 , user 1 requests deletion of data 1, and then the system uses data 3 with category 1.

$v, i \models e(\bar{t})$	iff $(e, \llbracket \bar{t} \rrbracket_v) \in D_i$	$v, i \models \neg \varphi$	iff $v, i \not\models \varphi$
$v, i \models \varphi \wedge \psi$	iff $v, i \models \varphi$ and $v, i \models \psi$	$v, i \models \exists x. \varphi$	iff $\exists d \in \mathbb{D}, i \models \varphi$ for some $d \in \mathbb{D}$
$v, i \models \bigcirc_I \varphi$	iff $v, i+1 \models \varphi$ and $\tau_{i+1} - \tau_i \in I$	$v, i \models \bullet_I \varphi$	iff $i > 0, v, i-1 \models \varphi$, and $\tau_i - \tau_{i-1} \in I$
$v, i \models \diamond_I \varphi$	iff $v, j \models \varphi$ for some $j \geq i$ with $\tau_j - \tau_i \in I$	$v, i \models \text{let } e(\bar{x}) = \varphi \text{ in } \psi$	iff $v, i \models_{\sigma[e \mapsto \varphi]} \psi$
$v, i \models \varphi S_I \psi$	iff $v, j \models \psi$ for some $j \leq i$ with $\tau_i - \tau_j \in I$ and $v, k \models \varphi$ for all $j < k \leq i$	$v, i \models x = c$	iff $v(x) = c$
$v, i \models \bar{y} \leftarrow \omega(\bar{s}; \bar{g})$	iff $v(\bar{y}) \in \hat{\omega}(\llbracket \bar{s} \rrbracket_{v'}, \bar{g})$, $\bar{g} = \{v' \mid \text{dom}(v') = \text{fv}(\varphi), v'(\bar{g}) = v(\bar{g}), v', i \models \varphi, 0 < \bar{g} < \infty\}$		

Fig. 1. MFOTL semantics for a fixed infinite trace σ . The free variables of an aggregation are $\bar{g} \cup \bar{y}$. $\langle\langle \cdot \cdot \cdot \rangle\rangle$ is a multiset, and $\sigma[e \mapsto \varphi]$ extends each D_j of σ with $\{e(\llbracket \bar{x} \rrbracket_{v'}) \mid \text{dom}(v') \supseteq \text{fv}(\varphi), v', j \models_{\sigma} \varphi\}$.

2.2 Metric first-order temporal logic (MFOTL)

MFOTL extends First-Order Logic (FOL) with temporal operators $\bigcirc$ ('next'), $\bullet$ ('previous'), $\diamond$ ('eventually') and S ('since') [45] equipped with intervals [31] that encode constraints on timestamps. We also include let bindings that introduce intermediate definitions, as well as SQL-style and user-defined aggregations. We thus follow Hublet et al. [28], except for the U ('until') operator being replaced by the $\diamond$ operator as a primitive, a deliberate restriction to the fragment we enforce.

Fix a set of variables $\mathbb{V}$, a set of *function symbols* $\mathbb{F}$ with arity $\iota : \mathbb{F} \rightarrow \mathbb{N}$, and a set *aggregation operator symbols* Ω with arity $\iota' : \Omega \rightarrow \mathbb{N}^2$. Each $f \in \mathbb{F}$ is interpreted by a fixed total function $\hat{f} : \mathbb{D}^{(\iota(f))} \rightarrow \mathbb{D}$ (e.g., arithmetic or string operations), and each $\omega \in \Omega$ is interpreted by a fixed total function $\hat{\omega}$ mapping finite multisets over $\mathbb{D}^n$ to finite subsets of $\mathbb{D}^m$ where $\iota'(\omega) = (n, m)$. The syntax of MFOTL is defined as follows:

$$t ::= x \mid c \mid f(\bar{t}) \quad \varphi ::= \top \mid e(\bar{t}) \mid x = c \mid \neg \varphi \mid \varphi \wedge \varphi \mid \exists x. \varphi \mid \bigcirc_I \varphi \mid \bullet_I \varphi \mid \diamond_I \varphi \mid \varphi S_I \varphi \mid \text{let } e(\bar{x}) = \varphi \text{ in } \varphi \mid \bar{x} \leftarrow \omega(\bar{t}; \bar{x}) \varphi,$$

where $e \in \mathbb{E}$, $x \in \mathbb{V}$, $c \in \mathbb{D}$, and $I \in \mathbb{I}$ ranges over non-empty intervals in $\mathbb{N}$. We use standard abbreviations $\perp \equiv \neg \top$, $\varphi \vee \psi \equiv \neg(\neg \varphi \wedge \neg \psi)$, $\varphi \rightarrow \psi \equiv \neg \varphi \vee \psi$, $\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$, $\forall x. \varphi \equiv \neg(\exists x. \neg \varphi)$, $\blacklozenge_I \varphi \equiv \top S_I \varphi$ ('once'), and $\Box_I \varphi \equiv \neg \diamond_I \neg \varphi$ ('always'). and omit $[0, \infty)$ intervals from the temporal operators' subscript. Furthermore, we write $\varphi[d/x]$ for the formula resulting from substituting the free variable x with the constant d in the formula φ .

Example 2.3. Suppose the time unit is *days*. Consider the following four formulae:

$$\begin{aligned} \varphi_{\text{law}} &= \Box(\forall c, d, u. \text{use}(c, d, u) \rightarrow (\neg \text{revoke}(u, c) S \text{consent}(u, c))) \\ \varphi_{\text{del}} &= \Box(\forall d, u. \text{deletion_request}(d, u) \rightarrow \diamond_{[0, 30]} \text{delete}(d, u)) \\ \varphi_{\text{let}} &= \text{let } \text{has_consent}(u, c) = \neg \text{revoke}(u, c) S \text{consent}(u, c) \text{ in } \Box(\forall c, d, u. \text{use}(c, d, u) \rightarrow \text{has_consent}(u, c)) \\ \varphi_{\text{agg}} &= n \leftarrow \text{CNT}(d; u) \blacklozenge_{[0, 10]} \text{use}(c, d, u) \end{aligned}$$

The first formula, φ_{law} , formalizes the *lawfulness of processing*: 'whenever data d with category c belonging to user u is processed, then u must have consented to the use of category c at some point in the past and not since revoked that consent.' The second formula, φ_{del} , formalizes the GDPR's *right to erasure*: 'whenever a user u requests the deletion of data d , then d must be deleted within 30 days.' The third formula, φ_{let} , illustrates *let bindings*: it defines $\text{has_consent}(u, c)$ as holding when u has consented to category c and not revoked that consent, and uses this definition to restate φ_{law} more concisely. The fourth formula, φ_{agg} , illustrates *aggregations*: it counts, grouped by u , the number of data items of user u used over the last 10 time units.

We write $\text{fv}(\varphi)$ for the set of free variables of a formula φ . MFOTL's formal semantics is defined in Figure 1. Given a valuation $v : \mathbb{V} \rightarrow \mathbb{D}$ whose domain includes $\text{fv}(\varphi)$, we define the interpretation of terms as $\llbracket x \rrbracket_v = v(x)$ (for variables), $\llbracket c \rrbracket_v = c$ (for constants), and $\llbracket f(\bar{t}) \rrbracket_v = \hat{f}(\llbracket \bar{t} \rrbracket_v)$ (for function applications). Each sequent $v, i \models_{\sigma} \varphi$ denotes that φ is satisfied at time-point i of trace σ under valuation v . We omit σ whenever it is clear from the context.

2.3 Enforcers

We consider a *proactive* runtime enforcement model [4, 27] where non-compliant traces can be corrected in two ways: by modifying existing event databases by causing or suppressing events (*reactively*); or by inserting a single event database before any timestamp increment (*proactively*). The latter feature allows enforcers to satisfy real-time deadlines by inserting events. We assume that a fixed subset of all events can be caused at any time ($\mathbb{C} \subseteq \mathbb{E}$), while another fixed subset can be suppressed ($\mathbb{S} \subseteq \mathbb{E}$). Formally, a *proactive enforcer* is a quadruple $\mathcal{E} = (\mathcal{S}, s_0, \mu, \nu)$ where $\mathcal{S}$ is a set of states, s_0 is an initial state, $\mu : \mathcal{S} \times \mathbb{T} \times \mathbb{N} \times \mathbb{DB} \rightarrow \mathcal{S} \times \mathbb{DB}_{\mathbb{C}} \times \mathbb{DB}_{\mathbb{S}}$ updates the state and causes and suppresses events when processing a database, and $\nu : \mathcal{S} \times \mathbb{T} \times \mathbb{N} \rightarrow \mathcal{S} \times (\mathbb{DB}_{\mathbb{C}} \cup \{\perp\})$ updates the state and optionally causes events (or does nothing) when a timestamp is incremented. As in [27, 28], the enforcer $\mathcal{E}$ modifies a trace ρ as follows:

Algorithm 1 Enforcement loop from Hublet et al. [27]

```

1: function  $\mathcal{E}(\rho = (\tau_i, D_i)_i)$ 
2:    $s \leftarrow s_0; \sigma \leftarrow \varepsilon$ 
3:   for  $i = 0, \dots, |\rho| - 1$  do
4:      $(s, C, S) \leftarrow \mu(s, \sigma, \tau_i, D_i); \sigma \leftarrow \sigma \cdot (\tau_i, (D_i \setminus S) \cup C)$  ▷ reactive
5:     for  $t \in \{\tau_i, \dots, \tau_{i+1} - 1\}$  (none if  $i = |\rho| - 1$ ) do
6:        $(s, C') \leftarrow \nu(s, \sigma, t);$  if  $C' \neq \perp$  then  $\sigma \leftarrow \sigma \cdot (t, C')$  end if ▷ proactive
7:     end for
8:   end for
9:   return  $\sigma$ 
10: end function

```

The enforcement loop incrementally processes a trace ρ of timestamped events (τ_i, D_i) , maintaining a current state s and an output trace σ . For each event, it first applies μ to update the state and transform the event's data (causing and suppressing event in C and S), and then calls ν before any timestamp increment to proactively cause any additional events.

An enforcer $\mathcal{E}$ is *sound* with respect to a closed formula φ if for any system trace $\sigma \in \mathbb{T}$, we have $\emptyset, 0 \models_{\mathcal{E}(\sigma)} \varphi$, i.e., the modified trace $\mathcal{E}(\sigma)$ complies with φ .

3 EF: An Operational Enforcement Language

We now present EF, a domain-specific language (DSL) that encodes the behavior of a proactive, first-order enforcer. EF programs are trace transducers that cause and suppress events when specific triggers hold, store temporal relations into tables, and perform fixpoint computations. One key feature of EF programs is that their per-time-point evaluation complexity can be easily bounded in terms of the size of some of their internal tables. If these tables remain small, this provides guarantees that the program's latency will not significantly grow with trace size, a key feature for deployment in long-running systems. In Havelund et al.'s terminology [22], EF is an *operational specification language* with a concrete executable semantics. We present EF's syntax (Section 3.1) and its semantics (Section 3.2) and analyze the complexity of executing EF programs (Section 3.3).

3.1 Syntax

We write *id* for identifiers, *n* for integer literals, and $\langle X \rangle$ for a comma-separated list of *X*. Figure 3 shows the full grammar of EF programs. Figure 2 shows the EF program enforcing the lawfulness policy φ_{law} of Example 2.3, which we revisit in Section 4, generating it automatically from φ_{law} .

An EF program is a sequence of *declarations* followed by a sequence of *items*. The declarations define the enforcement signature and auxiliary code: **event** *id*($\langle \tau \rangle$) declares an event with typed arguments; **pyinit** provides a Python preamble executed at start-up (imports and shared globals);

fun declares a Python function usable inside terms; and **tfun** declares a custom aggregation operator mapping a list of rows to another. The items define the enforcer's behavior:

- A **table** stores a persistent finite relation. At every iteration, the interpreter removes the rows matched by its **remove** clause and inserts those matched by its **add** clause. A **[window a b]** annotation makes the table *metric*: rows are active when they were inserted between a and b time units in the past ($b = *$ denotes ∞). A **lagged** table is consumed in the *next* iteration instead of the current one and cannot have a **remove** clause.
- A **let** defines a non-persistent relation within the current time-point. The **filter** keyword signals that the **let** expression may not be finite and can only be used as a boolean condition on top of another relation.
- An **agg let** evaluates its **over** clause, groups the results by **group_by**, and applies the operator *op* (**SUM**, **AVG**, ...) to each group.
- A **rule** matches its **trigger** to current events and, for each match, triggers an action. The action **+** *causes* an event, while **-** *suppresses* it. By default, the action is performed at the current time-point. With **+**, the **[delay n]** annotation triggers the event *proactively* in n time-units, while **[next n]** schedules it in n time-points.
- A **section** declaration groups all following rules (until the next **section**) into one evaluation block, run either **once** or to a local **fixpoint**, i.e., until the set of events obtained via suppressions and causations is stable. Every rule must belong to a section.

A *clause* is a pair of a disjunction π of *guards* $\kappa_1, \dots, \kappa_n, n \geq 0$, with each guard being a conjunction of atoms, and a *filter* **if** φ . An atom is either a predicate $e(t_1, \dots, t_k)$ or an equality $id == v$. Without a guard, the clause is reduced to its filter. If there is at least one guard, each free variable of the clause must be the argument of an atom of each guard. Guarded clauses can thus serve as queries over a relation: matching variable assignments are enumerated using the guards, and then filtered.

3.2 Semantics

Given an EF program, the EF reference interpreter, described in Algorithm 2, defines the behavior of the μ and ν functions defined in Section 2.

The *enforcer state* is a triple (T, T°, Ω) where $T : \mathbb{E} \rightarrow \mathcal{P}(\mathbb{N} \times \mathbb{D}^*)$ maps table names to (finite) timestamped relations, $T^\circ : \mathbb{E} \rightarrow \mathcal{P}(\mathbb{N} \times \mathbb{D}^*)$ buffers the content of lagged tables, and $\Omega \in \mathcal{P}(\mathbb{E} \times \mathbb{D}^* \times (\{\text{ts}, \text{tp}\} \times \mathbb{N}))$ accumulates proactive obligations created via **delay** or **next**, together with their deadlines. Initially, all tables are empty and $\Omega = \emptyset$.

Each **fun** declaration associates a function $\hat{f}$ to the function symbol f , and terms are evaluated as in MFOTL. Under an interpretation $R : \mathbb{E} \rightarrow \mathcal{P}(\mathbb{D}^*)$ of the predicates, guards and clauses evaluate to sets of valuations as follows: $\llbracket p(\bar{t}) \rrbracket_R = \{v \mid \text{dom}(v) = \text{fv}(\bar{t}), \llbracket \bar{t} \rrbracket_v \in R(p)\}$, $\llbracket x == c \rrbracket_R = \{\{x \mapsto c\}\}$, and $\llbracket \kappa_1 \text{ or } \dots \text{ or } \kappa_m \text{ if } \varphi \rrbracket_R = \{v \in \bigcup_i \bowtie_{Y \in \kappa_i} \llbracket Y \rrbracket_R \mid v \models_R \varphi\}$, where $\bowtie$ denotes a join.

Algorithm 2 defines the semantics of a program P with definitions $d_1, \dots, d_n$ of predicates $p_1, \dots, p_n$. The *working set* $\mathcal{S} = (T, \tau, D, C, S)$ consists of the tables, the current timestamp, the input events, and the events caused and suppressed so far. The tables contain *timestamped rows* of the form $\tau \cdot r$ where τ is a timestamp and r is a sequence of domain elements. The operator $\bar{s}^I = \{\bar{a} \mid \tau' \cdot \bar{a} \in s, \tau - \tau' \in I\}$ selects the rows of a timestamped relation whose timestamps are within the interval I . The notation $v \upharpoonright_{\bar{x}}$ denotes the row obtained by projecting v on $\bar{x}$, i.e., $(v(x_1), \dots, v(x_k))$.

The reference interpreter consists of four functions. EVAL evaluates definitions on the working set. INTERP extends $R_0 = (D \setminus S) \cup C$ with the value of each of the p_i , evaluating definitions in sequence using EVAL. UPDATE applies a single rule on the current working set, updating Ω . SATURATE applies rules section-wise (once for a **once** section, and until (Ω, C, S) reaches a fixed point for **fixpoint** sections), updates the persistent tables, and returns a new state. Throughout, $\mathcal{S}$ refers to the current

```

295 event use(int, int, int); // use(c, d, u)
296 event consent(int, int); // consent(u, c)
297 event revoke(int, int); // revoke(u, c)
298 event Sup_Exists0(int, int); event Sup_Exists2();
299 // [... standard library functions elided]
300 table Since1(c:int, u:int) := add {consent(u, c)} remove {revoke(u, c)};
301 let Exists2() := {use(c, d, u) if !Since1(c, u)};
302 section fixpoint;
303 rule +Sup_Exists2() := trigger {if Exists2()};
304 rule +Sup_Exists0(c, u) := trigger {Sup_Exists2() & use(c, d, u) if !Since1(c, u)};
305 rule -use(c, d, u) := trigger {Sup_Exists0(c, u) & use(c, d, u)};

```

Fig. 2. EF program generated by ENFLASH for φ_{law} (Example 2.3): table Since1 implements the S subformula, obligation events Sup_Existsx discharge existentials, the last rule suppresses use(c, d, u) when consent fails.

```

309 prog ::= evdecl* [pyinit] fun* tfun* item*
310 evdecl ::= event id(<τ>);
311 fun ::= fun id(<c>) : τ {python}
312 item ::= table | let | agg | rule | sec
313 table ::= [ℓ] [lagged] table id(<c>) [ [window n b] ] := add {cl} [ remove {cl} ];
314 let ::= [ℓ] [filter] let id(<c>) := {cl};
315 agg ::= [ℓ] agg let id(<c>) := id(<t>) group_by <id> over {cl};
316 rule ::= [ℓ] rule (+ | -) id(<t>) [ [delay n] ] [ [next n] ] := trigger {cl};
317 cl ::= π [if φ] | if φ π ::= κ (or κ)*
318 φ ::= true | false | id(<t>) | φ & φ | φ | φ | !φ
319 τ ::= int | float | str | bool
320
321 pyinit ::= pyinit {python}
322 tfun ::= tfun id {python}
323 sec ::= section (fixpoint | once);
324
325 κ ::= γ (& γ)* γ ::= id(<id | v>) | id == v
326 t ::= id | const | id(<t>)
327 c ::= id : τ ℓ ::= @id b ::= n | *

```

Fig. 3. Grammar of EF programs

values of T , C and S . The reactive function μ simply calls SATURATE after retrieving the **next** obligations due at the current time-point. The proactive ν function first computes the events to be caused at the current timestamp or time-point, identifying obligations that hit their deadline. Then, if and only there exist any such events, it calls the SATURATE loop.

Example 3.1. We run the program of Figure 2 on the trace σ_1 of Example 2.2. The initial state is $(T, T^\circ, \Omega) = (\{\text{Since1} \mapsto \emptyset\}, \emptyset, \emptyset)$. Since the program has no **delay** or **next** rules, Ω remains empty and ν always returns $\perp$.

At time-point 0, we have $R_0 = \{\text{consent} \mapsto \{(1, 1)\}\}$, then $\text{INTERP}(S)(\text{Since1}) = \emptyset \cup \{(1, 1)\}$ and $\text{INTERP}(S)(\text{Exists2}) = \emptyset$ as there is no use. No trigger matches, hence the fixpoint stops after one pass with $C = S = \emptyset$, and μ outputs D . The SATURATE function commits $T(\text{Since1}) = \{10 \cdot (1, 1)\}$.

At time-point 1, we still have $\text{INTERP}(S)(\text{Since1}) = \{(1, 1)\}$. Since user 1 never consented to category 2, $\text{INTERP}(S)(\text{Exists2}) = \{\emptyset\}$. The first iteration of the fixpoint proceeds as follows:

- (1) The rule **+Sup_Exists2()** is activated, yielding $C = \{\text{Sup_Exists2}()\}$;
- (2) The trigger of **+Sup_Exists0(c, u)** joins **Sup_Exists2()** with **use** and filters by **!Since1(c, u)**, obtaining the valuation $v = \{c \mapsto 2, d \mapsto 1, u \mapsto 1\}$; hence **Sup_Exists0(2, 1)** is added to C ;
- (3) The rule **-use(c, d, u)** matches **Sup_Exists0(2, 1)** with **use(2, 1, 1)**, yielding $S = \{\text{use}(2, 1, 1)\}$.

In the second iteration of the fixpoint, $\text{use}(2, 1, 1) \notin R_0 = (D \setminus S) \cup C$ since $\text{use}(2, 1, 1) \in S$, and hence $\text{INTERP}(S)(\text{Exists2}) = \emptyset$. The set (Ω, C, S) is unchanged and the fixpoint stops. The output is $(D \setminus S) \cup C = \{\text{use}(1, 3, 1), \text{Sup_Exists2}(), \text{Sup_Exists0}(2, 1)\}$; the unlawful use is suppressed.

Algorithm 2 EF interpreter for a program P with predicates $p_1, \dots, p_n$ and definitions $d_1, \dots, d_n$

```

1: function  $\mu((T, T^\circ, \Omega), \sigma, \tau, D)$ 
2:    $O \leftarrow \{e \mid (e, (\text{tp}, |\sigma|)) \in \Omega\}$ 
3:    $(T, C, S, \Omega) \leftarrow \text{SATURATE}$ 
4:      $((T, \tau, D, O, \emptyset), T^\circ, \Omega, \sigma)$ 
5:   return  $((T, T^\circ, \Omega), C, S)$ 
6:
7: function  $\nu((T, T^\circ, \Omega), \sigma, \tau)$ 
8:    $C \leftarrow \{e \mid (e, (\text{tp}, |\sigma|)) \in \Omega$ 
9:      $\vee (e, (\text{ts}, \tau)) \in \Omega\}$ 
10:  if  $C = \emptyset$  then return  $((T, T^\circ, \Omega), \perp)$ 
11:   $(T, C, S, \Omega) \leftarrow \text{SATURATE}$ 
12:     $((T, \tau, \emptyset, C, \emptyset), T^\circ, \Omega, \sigma)$ 
13:  return  $((T, T^\circ, \Omega), C)$ 
14:
15: function  $\text{SATURATE}(S = (T, \tau, D, C, S), T^\circ, \Omega, \sigma)$ 
16:  for all section  $s$  of  $P$  in order do
17:    repeat
18:      for all rule  $r \in s$  do
19:         $(\Omega, C, S) \leftarrow \text{UPDATE}(r, \Omega, S, \sigma)$ 
20:      until  $(\Omega, C, S)$  unchanged or  $s$  is once
21:
22:  for all table  $p_i(\bar{x}_i) := \text{add } \{a_i\} \text{ remove } \{r_i\}$  do
23:     $T(p_i) \leftarrow (T(p_i) \setminus \{\tau' \cdot v \upharpoonright_{\bar{x}_i} \mid$ 
24:       $v \in \llbracket r_i \rrbracket_{\text{INTERP}(S)}, \tau' \in \mathbb{N}\})$ 
25:     $\cup \{\tau \cdot v \upharpoonright_{\bar{x}_i} \mid v \in \llbracket a_i \rrbracket_{\text{INTERP}(S)}\}$ 
26:
27:  for all lagged table  $p_i(\bar{x}_i) := \text{add } \{c_i\}$  do
28:     $T^\circ(p_i) \leftarrow \{\tau \cdot v \upharpoonright_{\bar{x}_i} \mid v \in \llbracket c_i \rrbracket_{\text{INTERP}(S)}\}$ 
29:
30:  for all lagged table  $p_i$  do
31:     $T(p_i) \leftarrow T^\circ(p_i)$ 
32:  return  $(T, C, S, \Omega)$ 
33:
34: function  $\text{INTERP}(S = (T, \tau, D, C, S))$ 
35:   $R_0 \leftarrow (D \setminus S) \cup C$ 
36:  for  $i = 1, \dots, n$  do
37:     $R_i \leftarrow R_{i-1}[p_i \mapsto \text{EVAL}(d_i, R_{i-1}, T, \tau)]$ 
38:  return  $R_n$ 
39:
40: function  $\text{EVAL}(d, R, T, \tau)$ 
41:  if  $d = \text{let } p(\bar{x}) := \{c\}$  then
42:    return  $\{v \upharpoonright_{\bar{x}} \mid v \in \llbracket c \rrbracket_R\}$ 
43:  else if  $d = \text{table } p(\bar{x}) [\text{window } n \ b] :=$ 
44:     $\text{add } \{a\} \text{ remove } \{r\}$  then
45:    return  $(\overline{T(p)})^{[n, b]} \setminus \{v \upharpoonright_{\bar{x}} \mid v \in \llbracket r \rrbracket_R\}$ 
46:     $\cup (n = 0 ? \{v \upharpoonright_{\bar{x}} \mid v \in \llbracket a \rrbracket_R\} : \emptyset)$ 
47:  else if  $d = \text{lagged table } p(\bar{x}) [\text{window } n \ b] :=$ 
48:     $\text{add } \{c\}$  then
49:    return  $\overline{T(p)}^{[n, b]}$ 
50:  else if  $d = \text{agg let } p(\bar{g}, \bar{o}) := g(\bar{s})$ 
51:    group\_by  $\bar{g}$  over  $\{c\}$  then
52:    return  $\{\bar{v}(\bar{g}) \cdot \bar{w} \mid \bar{v} \in \llbracket c \rrbracket_R,$ 
53:       $\bar{w} \in \bar{g} \llbracket \bar{s} \rrbracket_v \mid v \in \llbracket c \rrbracket_R, v(\bar{g}) = \bar{v}(\bar{g}) \gg\}$ 
54:
55: function  $\text{UPDATE}(r, \Omega, S = (T, \tau, D, C, S), \sigma)$ 
56:   $// r = \text{rule } \alpha p(\bar{i}) \dots := \text{trigger } \{c\}$ 
57:   $A \leftarrow \{\llbracket \bar{i} \rrbracket_v \mid v \in \llbracket c \rrbracket_{\text{INTERP}(S)}\}$ 
58:  if  $r$  has [delay N] then
59:    return  $(\Omega \cup \{(p(\bar{a}), (\text{ts}, \tau + N))$ 
60:       $\mid \bar{a} \in A\}, C, S)$ 
61:  else if  $r$  has [next N] then
62:    return  $(\Omega \cup \{(p(\bar{a}), (\text{tp}, |\sigma| + N))$ 
63:       $\mid \bar{a} \in A\}, C, S)$ 
64:  else if  $\alpha = +$  then
65:    return  $(\Omega, C \cup \{p(\bar{a}) \mid \bar{a} \in A\}, S)$ 
66:  else
67:    return  $(\Omega, C, S \cup \{p(\bar{a}) \mid \bar{a} \in A\})$ 

```

3.3 Complexity

We bound the runtime of executing a program P in terms of three parameters: the largest number of guards in any single conjunction κ of any clause of P , which we denote w ; the largest number of tuples in any relation $\mathcal{S}(p)$ occurring in at least one guard, which we denote N ; and the total number of fixpoint iterations performed, which we denote K . We make the following assumptions that let us isolate relational work:

- (A1) the original trace carries $O(1)$ events per time-point;
- (A2) each scalar function $\hat{f}$, comparison, and aggregation step is $O(1)$, membership tests, insertion, and deletion within a relation is amortized $O(1)$ with hash maps, and any $\hat{w}$ on m input rows runs in $O(m)$, returning $O(m)$ rows.

Under (A1) the input-event relations have size $O(1)$, so N is governed by the tables. We prove:

LEMMA 3.2. *The boolean $\varphi, v \models_S \varphi$ can be computed in $O(1)$.*

PROOF. Since φ has constant size, so it suffices that each atom is $O(1)$. A comparison $t_1 \bowtie t_2$ evaluates its constant-size terms under v in $O(1)$ by (A2). A predicate a let-bound atom $p(\bar{i})$ involves the membership test $\llbracket \bar{i} \rrbracket_v \in \mathcal{S}(p)$, which costs $O(1)$ using hashsets. For a table, membership in

$\mathcal{W}(T(p))$ can be checked in $O(1)$ by maintaining two separate subtables (timestamps in $\tau - [a, b]$ and in $\tau - [0, a)$) and moving elements between subtables on update. $\square$

LEMMA 3.3. *Given R_{i-1} , computing $\llbracket c \rrbracket_{R_{i-1}}$ for any clause c with all identifiers in the domain of R_{i-1} takes $O(N^w)$ time.*

PROOF. Matching a conjunction joins up to guards, each ranging over a relation with at most N tuples. The join enumerates at most N^w valuations, each requiring an $O(1)$ filter by Lemma 3.2. $\square$

LEMMA 3.4. *Evaluating one rule of P takes $O(N^w)$ time.*

PROOF. With memoization, each relation the trigger depends on is computed at most once, in $O(N^w)$ by Lemma 3.3. Matching the trigger and instantiating its up to N^w heads are $O(N^w)$. $\square$

THEOREM 3.5. *One call to μ or ν takes $O(K \cdot N^w)$ time.*

PROOF. Each of the K iterations is one pass over the $O(1)$ rules of a section, each rule $O(N^w)$ by Lemma 3.4, hence $O(N^w)$ per iteration. Window updates touch each tuple $O(1)$ times, committing the layer recomputes constantly many table relations ($O(N^w)$). Each emitted obligation—at most $O(K \cdot N^w)$ per time-point—is discharged once by ν in $O(1)$ by evaluating the validation filter. $\square$

THEOREM 3.6. *A trace of L time-points is processed in $O(L \cdot K_{\max} \cdot N_{\max}^w)$, where $K_{\max}$ and $N_{\max}$ are the largest iteration count and relation size over all time-points.*

PROOF. Sum the complexity of Theorem 3.5 over the L time-points. $\square$

Note that if no tables are used in guards, such as in Example 2, then $N_{\max} = O(1)$ by (A1). If P can be shown to always reach its fixed points in a bounded number of steps, this ensures that the total complexity is $O(L)$, an amortized constant per-step complexity. This is particularly desirable in long-running systems where large enforcer states can accumulate and penalize evaluation latency.

4 From Enforceable MFOTL to EF

In this section, we present an algorithm that generates EF programs enforcing formulae in an expressive fragment of MFOTL. Our generation algorithm consists of four components: a normalization procedure that places all temporal subformulae of a formula behind let bindings (4.1); rewrite rules that extract guards from subformulae (4.2); rewrite rules that generate enforceable clauses from a formula (4.3); and a dependency analysis that decides enforceability of a conjunction of clauses and fixes their evaluation order (4.5).

4.1 Let-normal form

In this section, we work with MFOTL formulae rewritten into *let-normal form*. Let-normal form has the shape

$$\text{let } e_1(\bar{x}_1) = \varphi_1 \text{ in } \dots \text{let } e_m(\bar{x}_m) = \varphi_m \text{ in } \square \chi_1 \wedge \dots \wedge \square \chi_n,$$

where each φ_i is one of ψ , $\bullet_I \psi$, $\psi_I S_I \psi_r$, or $\bar{y} \leftarrow \omega(\bar{t}; \bar{g}) \psi$ and ψ is $\exists y_1, \dots, y_k. \chi$, with χ an MFOTL formula without any quantifiers (except over subformulae with future operators), $\bullet$ and S_I operators, or aggregation/table operators. With formulae in let-normal form, quantification, past-temporal, and aggregation/table operators are applied as the last step of evaluating a let binding. One shows:

THEOREM 4.1. *For any formula $\square \varphi$ in MFOTL, there exists an equivalent ψ in let-normal form.*

[$\sqsubseteq_{\text{EF}}$ let_normal_form]

$$\begin{array}{c}
\text{NONE} \frac{}{m \vdash \Phi \rightsquigarrow_0^p (\{\top\}, \Phi)} \quad \text{VAC} \frac{}{m \vdash \top \rightsquigarrow_X^- (\emptyset, \top)} \quad \text{PRED} \frac{p \in m \quad X \subseteq \bar{t}}{m \vdash p(\bar{t}) \rightsquigarrow_X^+ (\{p(\bar{t})\}, \top)} \\
\text{EQ} \frac{X \subseteq \{x\}}{m \vdash x = c \rightsquigarrow_X^+ (\{x = c\}, \top)} \quad \text{AND}^+ \frac{m \vdash \varphi \rightsquigarrow_{X_1}^+ (\pi_1, \varphi') \quad m \vdash \psi \rightsquigarrow_{X_2}^+ (\pi_2, \psi') \quad X \subseteq X_1 \cup X_2}{m \vdash \varphi \wedge \psi \rightsquigarrow_X^+ (\{\kappa_1 \wedge \kappa_2 \mid \kappa_1 \in \pi_1, \kappa_2 \in \pi_2\}, \varphi' \wedge \psi')} \\
\text{AND}^- \frac{m \vdash \varphi \rightsquigarrow_X^- (\pi_1, \varphi') \quad m \vdash \psi \rightsquigarrow_X^- (\pi_2, \psi')}{m \vdash \varphi \wedge \psi \rightsquigarrow_X^- (\pi_1 \cup \pi_2, (\pi_1 \rightarrow \varphi') \wedge (\pi_2 \rightarrow \psi'))} \quad \text{NEG} \frac{m \vdash \varphi \rightsquigarrow_X^{-p} (\pi, \varphi')}{m \vdash \neg \varphi \rightsquigarrow_X^p (\pi, \neg \varphi')}
\end{array}$$

Fig. 4. Joint guard extraction

4.2 Guard extraction

Consider a quantifier-free FOL formula Φ . Next, we show how to derive a disjunction of guards $\pi = \bigvee_i \bigwedge_j \gamma_{ij}$ and a filter φ such that $\pi \wedge \varphi \equiv \Phi$ and every guard κ in π binds every variable x of a given set X , i.e., κ contains an atom $p(\bar{t})$ with $x \in \bar{t}$ or an atom $x = c$.

Given a set m of enumerable events that can be used as guards, we define a transformation $m \vdash \Phi \rightsquigarrow_X^+ (\pi, \varphi')$ that finds a guard for every $x \in X$ in Φ and collects the resulting *joint guard* of X in π , preserving $\Phi \equiv \pi \wedge \varphi$. The rules defining $\rightsquigarrow_X^+$ and its dual $\rightsquigarrow_X^-$ are shown in Figure 4.

Rules PRED and EQ insert a new guard from an enumerable event and an equation, respectively. VAC provides the base case for $\top$. Rule NEG exchanges $\rightsquigarrow_X^+$ for $\rightsquigarrow_X^-$. Rules AND⁺ and AND⁻ extract guards from conjuncts. The former relies on the equation $\pi \wedge \varphi_j \equiv \pi' \wedge \varphi'_j \implies \pi \wedge \bigwedge_i \varphi_i \equiv \pi' \wedge \bigwedge_{i \neq j} \varphi_i \wedge \varphi'_j$, which enables a guard to be extracted from any conjunct. The latter uses the dual equation $\forall i. \pi \rightarrow \varphi_i \equiv \pi'_i \rightarrow \varphi'_i \implies \pi \rightarrow \bigwedge_i \varphi_i \equiv \bigvee_i \pi'_i \rightarrow \bigwedge_i (\pi'_i \rightarrow \varphi'_i)$. We write $\text{Guards}_X^m(\Phi) = (\pi, \varphi)$ if $m \vdash \Phi \rightsquigarrow_X^+ (\pi, \varphi)$ (for a fixed choice among several derivations), and $\text{Guards}_X^m(\Phi) = \perp$ if no such derivation exists and show:

LEMMA 4.2. *If $\text{Guards}_X^m(\Phi) = (\pi, \varphi)$, then every $\kappa \in \pi$ binds every $x \in X$, and $\bigvee_{\kappa \in \pi} \kappa \wedge \varphi \equiv \Phi$.*
[LBVN Lemma_4_2]

For a trigger (π, ψ) and a variable x , we write $m \vdash (\pi, \psi) \rightsquigarrow_x^p (\pi', \psi')$ if either every $\kappa \in \pi$ binds x and $(\pi', \psi') = (\pi, \psi)$, or $m \vdash \psi \rightsquigarrow_{\{x\}}^p (\pi_0, \psi')$ and $\pi' = \{\kappa \wedge \kappa_0 \mid \kappa \in \pi, \kappa_0 \in \pi_0\}$.

4.3 Enforcement rewriting

Our rewrite rules turns a formula into a set of *enforcement clauses*. A clause $\theta \Rightarrow \varepsilon$ pairs a trigger $\theta = (\pi, \varphi)$ with an *effect* ε of the form $e(\bar{t})$, $\neg e(\bar{t})$, $\diamond_I e(\bar{t})$, or $\bigcirc \dots \bigcirc e(\bar{t})$. A clause denotes the rule $\square \forall \bar{x}. \theta \rightarrow \varepsilon$ with $\bar{x} = \text{fv}(\theta) \cup \text{fv}(\varepsilon)$: for any valuation satisfying the trigger, the effects are applied.

The judgement $\Gamma \vdash \varphi \hookrightarrow^C C$ produces a set of sets of clauses $C = \{C_k\}_k$ such that enforcing *any* alternative C_k enforces φ . Dually, the judgement $\Gamma \vdash \varphi \hookrightarrow^S C$ is such that enforcing any alternative enforces $\neg \varphi$. The environment Γ maps let-bound event names to a triple of booleans (g, c, s) that indicate whether the corresponding event is guarded, causable, and suppressable, respectively.

Let $C[f]$ denote $\{\{f(\pi, \psi, \varepsilon) \mid (\pi, \psi) \Rightarrow \varepsilon \in C\} \mid C \in C\}$ and $C_1 \otimes C_2$ denote $\{C_1 \cup C_2 \mid (C_1, C_2) \in C_1 \times C_2\}$. The enforcement rewrite rules are shown in Figure 5. We write m_Γ for the set of enumerable events: the base events and the let-bound e with $\Gamma(e)_1 = \top$.

The two Ev rules are the only rules implementing immediate causation or suppression. If the signature declares e causable ($e \in \mathbb{C}$), then e is enforced by an unconditional clause that causes $e(\bar{t})$; if it declares e suppressable ($e \in \mathbb{S}$), the clause fires exactly when $e(\bar{t})$ is present and suppresses it. The LET rules assert a fresh *obligation event* Cau_e (resp. Sup_e), of the same arity as e , that will be consumed by clauses that realize e 's body. The NEG rule flips the target formula's polarity: causing $\neg \varphi$ is suppressing φ ($\bar{\mathbb{C}} = \mathbb{S}$, $\bar{\mathbb{S}} = \mathbb{C}$). To cause a conjunction (AND^C), every conjunct must be caused,

$$\begin{array}{c}
\text{491} \quad \text{492} \quad \text{493} \quad \text{494} \quad \text{495} \quad \text{496} \quad \text{497} \quad \text{498} \quad \text{499} \quad \text{500} \quad \text{501} \quad \text{502} \quad \text{503} \quad \text{504} \quad \text{505} \quad \text{506} \quad \text{507} \quad \text{508} \quad \text{509} \quad \text{510} \quad \text{511} \quad \text{512} \quad \text{513} \quad \text{514} \quad \text{515} \quad \text{516} \quad \text{517} \quad \text{518} \quad \text{519} \quad \text{520} \quad \text{521} \quad \text{522} \quad \text{523} \quad \text{524} \quad \text{525} \quad \text{526} \quad \text{527} \quad \text{528} \quad \text{529} \quad \text{530} \quad \text{531} \quad \text{532} \quad \text{533} \quad \text{534} \quad \text{535} \quad \text{536} \quad \text{537} \quad \text{538} \quad \text{539} \\
\begin{array}{l}
\text{LET}^C \frac{}{\Gamma \vdash \tau \hookrightarrow^C \{\emptyset\}} \quad \text{Ev}^C \frac{e \in \mathbb{C}}{\Gamma \vdash e(\bar{i}) \hookrightarrow^C \{\{(\tau, \tau) \Rightarrow e(\bar{i})\}\}} \quad \text{Ev}^S \frac{e \in \mathbb{S}}{\Gamma \vdash e(\bar{i}) \hookrightarrow^S \{\{(e(\bar{i}), \tau) \Rightarrow \neg e(\bar{i})\}\}} \\
\text{LET}^C \frac{\Gamma(e)_2 = \tau}{\Gamma \vdash e(\bar{i}) \hookrightarrow^C \{\{(\tau, \tau) \Rightarrow \text{Cau}_e(\bar{i})\}\}} \quad \text{LET}^S \frac{\Gamma(e)_3 = \tau}{\Gamma \vdash e(\bar{i}) \hookrightarrow^S \{\{(e(\bar{i}), \tau) \Rightarrow \text{Sup}_e(\bar{i})\}\}} \\
\text{NEG} \frac{\Gamma \vdash \varphi \hookrightarrow^i C}{\Gamma \vdash \neg \varphi \hookrightarrow^t C} \quad \text{AND}^S \frac{\Gamma \vdash \varphi_j \hookrightarrow^S C_j \quad \varphi_i \text{ present for } i \neq j}{\Gamma \vdash \bigwedge_i \varphi_i \hookrightarrow^S C_j[\pi, \psi, \varepsilon \mapsto (\pi, \psi \wedge \bigwedge_{i \neq j} \varphi_i) \Rightarrow \varepsilon]} \\
\text{AND}^C \frac{\forall i. \Gamma \vdash \varphi_i \hookrightarrow^C C_i}{\Gamma \vdash \bigwedge_i \varphi_i \hookrightarrow^C \bigotimes_i C_i} \quad \text{EX}^C \frac{\Gamma \vdash \varphi \hookrightarrow^C C}{\Gamma \vdash \exists x. \varphi \hookrightarrow^C C[\pi, \psi, \varepsilon \mapsto (\pi[0/x], \psi[0/x]) \Rightarrow \varepsilon[0/x]]} \\
\text{EX}^S \frac{\Gamma \vdash \varphi \hookrightarrow^S C \quad C' = \{C \in \mathbb{C} \mid \forall (\pi, \psi) \Rightarrow \varepsilon \in C. \exists \pi', \psi'. (\pi, \psi) \rightsquigarrow_x^+ (\pi', \psi')\}}{\Gamma \vdash \exists x. \varphi \hookrightarrow^S \{\{(\pi', \psi') \Rightarrow \varepsilon \mid (\pi, \psi) \Rightarrow \varepsilon \in C, (\pi, \psi) \rightsquigarrow_x^+ (\pi', \psi')\} \mid C \in C'\}} \\
\text{FUT}_{\diamond}^C \frac{\Gamma \vdash \varphi \hookrightarrow^C C \quad C' = \{C \in \mathbb{C} \mid C \neq \emptyset \wedge \forall (\pi, \psi) \Rightarrow \varepsilon \in C. \exists p, \bar{t}. \varepsilon = p(\bar{t}) \wedge \pi = \psi = \tau\} \quad 1 \leq b < \infty}{\Gamma \vdash \diamond_{[a,b]} \varphi \hookrightarrow^C C'[\pi, \psi, \varepsilon \mapsto (\tau, \tau) \Rightarrow \diamond_{[b,b]} \varepsilon]} \\
\text{FUT}_{\circ}^C \frac{\Gamma \vdash \varphi \hookrightarrow^C C \quad 1 \leq b \quad C' = \{C \in \mathbb{C} \mid C \neq \emptyset \wedge \forall (\pi, \psi) \Rightarrow \varepsilon \in C. \exists p, \bar{t}. \varepsilon = p(\bar{t}) \wedge \pi = \psi = \tau\}}{\Gamma \vdash \circ_{[0,b]} \varphi \hookrightarrow^C C'[\pi, \psi, \varepsilon \mapsto (\tau, \tau) \Rightarrow \circ \varepsilon]} \\
\text{FUT}_{\circ^n}^C \frac{\Gamma \vdash \varphi \hookrightarrow^C C \quad 1 \leq n \quad C' = \{C \in \mathbb{C} \mid \forall (\pi, \psi) \Rightarrow \varepsilon \in C. \exists p, \bar{t}. \varepsilon = p(\bar{t}) \wedge \pi = \psi = \tau\}}{\Gamma \vdash \underbrace{\circ \dots \circ}_n \varphi \hookrightarrow^C C'[\pi, \psi, \varepsilon \mapsto (\tau, \tau) \Rightarrow \circ \dots \circ \varepsilon]}
\end{array}
\end{array}$$

Fig. 5. Enforcement rewrite rules

so the clause sets are combined by the product $\bigotimes$; to suppress it (AND^S) it suffices to falsify one conjunct φ_j , with the other conjuncts added to the filter. To cause $\exists x. \varphi$ (EX^C), the rule substitutes a canonical value 0 for x . To suppress it (EX^S), every witness must be falsified, hence x must be guarded.

The temporal rules implement deferred causation. The $\text{FUT}_{\diamond}$ and $\text{FUT}_{\circ}$ rules apply only when the sub-derivation has reduced the body to a single causable event; the effect is scheduled at b resp. after n time points. Since ν only inserts a time-point when something is due, the intermediate time-points of a chain of $\circ$ operators need not exist within a bounded interval; hence bounded intervals are only supported for a single $\circ$ ($\text{FUT}_{\circ}$), whereas chains must be unbounded ($\text{FUT}_{\circ^n}$). Deadlines are at least one time unit (resp. time-point) ahead, so that obligations created by ν are discharged by a later call.

4.4 Generating the candidate clause sets

We write $\text{gate}_p^C(C)$ (resp. gate_p^S) for the set of sets of clauses obtained by conjoining the atom $\text{Cau}_p(\bar{x})$ (resp. Sup_p) to every guard π in C , and $\Gamma \vdash \varphi \hookrightarrow^\alpha C$ for the set of all $C \in \mathbb{C}$ with $\Gamma \vdash \varphi \hookrightarrow^\alpha C$. Rewriting is described in Algorithm 3. The algorithm computes a let-normal form $(\mathcal{L}, \varphi^b)$ of $\square \varphi$ as in Section 4.1, with $\mathcal{L}$ its lets and $\varphi^b = \chi_1 \wedge \dots \wedge \chi_n$ the conjunction of its $\square$ -bodies. It first processes every let in order, generating the triple $\Gamma(p) = (g, c, s)$ and causation/suppression clauses $C^C(p), C^S(p)$, and then rewrites φ^b . The output is a *set of candidate clause sets*; to yield a valid enforcer, a clause set will have to pass the tests in Section 4.5.

TYPELET performs a case split on the let body's outermost operator. Each case decides *guardedness* g and computes the two clause families by rewriting the relevant operand and gating it with p 's obligation event. The boolean c (resp. s) can be computed by checking the emptiness of the set of causation (resp. suppression) clauses. Which clauses are generated depends on the topmost operator of the let. A $\diamond$ or S is causable only when its interval admits the present ($a = 0$) by causing the (right) operand; a S is suppressable when either $a > 0$ and its left operand is suppressable or both its operands are. A $\bullet$ or aggregation is neither causable or suppressable. Insufficient guarding

Algorithm 3 Generation of candidate clause sets

```

1: function GENERATE( $\square \varphi$ )
2:    $(\mathcal{L}, \varphi^b) \leftarrow \text{LETNORMALFORM}(\square \varphi); \quad \Gamma \leftarrow \emptyset$ 
3:   for all  $(p(\bar{x}) := \psi) \in \mathcal{L}$  do  $\Gamma \leftarrow \text{TYPELET}(\Gamma, p, \bar{x}, \psi)$ 
4:   return  $\bigcup_{C \in \mathcal{C}, \Gamma \vdash \varphi^b \hookrightarrow C} \text{REALIZATIONS}(\Gamma, C)$ 
5: end function

6: function TYPELET( $\Gamma, p, \bar{x}, \psi$ )
7:    $\chi \leftarrow \text{STRIP EXISTS}(\psi); m \leftarrow m_\Gamma; C^C(p), C^S(p) \leftarrow \emptyset$ 
8:   switch  $\chi$  do
9:     case  $\blacklozenge_{[a,b]} \varphi$ 
10:       $\theta \leftarrow \text{Guards}_{\text{fv}(\varphi)}^m(\varphi);$  if  $\theta = \perp$  then reject end if;  $C^C(p) \leftarrow a = 0 ? \text{gate}_p^C(\Gamma \vdash \varphi \hookrightarrow^C) : \emptyset$ 
11:     case  $\bullet \varphi \mid \text{Agg}(\dots, \varphi)$ 
12:       $\theta \leftarrow \text{Guards}_{\text{fv}(\varphi)}^m(\varphi);$  if  $\theta = \perp$  then reject end if
13:     case  $\varphi_l S_{[a,b]} \varphi_r$ 
14:       $\theta_l \leftarrow \text{Guards}_{\bar{x} \cup \text{fv}(\chi)}^m(\neg \varphi_l); \theta_r \leftarrow \text{Guards}_{\bar{x} \cup \text{fv}(\chi)}^m(\varphi_r)$ 
15:      if  $\theta_l = \perp \vee \theta_r = \perp$  then reject end if
16:      if  $a = 0$  then  $C^C(p) \leftarrow \text{gate}_p^C(\Gamma \vdash \varphi_r \hookrightarrow^C); C^S(p) \leftarrow \text{gate}_p^S((\Gamma \vdash \varphi_l \hookrightarrow^S) \otimes (\Gamma \vdash \varphi_r \hookrightarrow^S))$ 
17:      else  $C^S(p) \leftarrow \text{gate}_p^S(\Gamma \vdash \varphi_l \hookrightarrow^S)$  end if
18:     case otherwise
19:       $\theta \leftarrow \text{Guards}_{\bar{x} \cup \text{fv}(\chi)}^m(\chi);$  if  $\theta = \perp$  then return  $\Gamma[p \mapsto (\perp, \perp, \perp)]$  end if
20:      if  $\chi$  is present then  $C^C(p) \leftarrow \text{gate}_p^C(\Gamma \vdash \psi \hookrightarrow^C); C^S(p) \leftarrow \text{gate}_p^S(\Gamma \vdash \psi \hookrightarrow^S)$  end if
21:   return  $\Gamma[p \mapsto (\top, C^C(p) \neq \emptyset, C^S(p) \neq \emptyset)]$ 
22: end function

```

is fatal for a temporal or aggregation let. For any other let, it downgrades p to *filter-only*, making it usable as a boolean condition but not caused or suppressed.

Once Γ is complete, GENERATE rewrites φ^b at $\mathbb{C}$, yielding a set of candidate clause sets. REALIZATIONS(Γ, C) is the set of the least clause sets $R \supseteq C$ such that, for some choice of $f(p) \in C^C(p)$ and $g(p) \in C^S(p)$ for every let p , $f(p) \subseteq R$ (resp. $g(p) \subseteq R$) whenever the effect of a clause of R is on Cau_p (resp. Sup_p), possibly under $\diamond$ or $\bigcirc$. The resulting candidate clause sets serve as input to the dependency analysis in the next section, which filters clause sets yielding a terminating enforcer.

4.5 Dependency analysis

Not all candidate clause sets generated in the previous step yield a sound enforcer. For example, the set $C_1 = \{(\top, \top) \Rightarrow A(0), (\top, \top) \Rightarrow \neg A(0)\}$ is unsatisfiable, while $C_2 = \{(A(x), \top) \Rightarrow A(x+1)\}$ generates an infinite number of events and never terminates. We state two checks that together guarantee that a set of clauses can be finitely enforced. The first rules out events that are simultaneously caused and suppressed, as in C_1 ; the second rules out C_2 's non-terminating fixpoints.

Cause/suppress conflicts. Consider the *Event Dependency Graph* (EDG) whose vertices are the base and obligation events (let atoms are decomposed into the events defining them), and there is an edge $e \rightarrow e'$ labeled with c_k^a when e occurs in the trigger of some clause k and e' occurs in its effect with polarity $a \in \{\mathbb{C}, \mathbb{S}\}$ (deferred effects have polarity $\mathbb{C}$) (Figure 6). Its strongly connected components (SCCs) are mutually-influencing event sets. For each e that one clause with trigger $\theta_{\mathbb{C}}$ causes and another clause with trigger $\theta_{\mathbb{S}}$ suppresses, the pair $(\theta_{\mathbb{C}}, \theta_{\mathbb{S}})$ is sent to an SMT solver, which tests the unsatisfiability of $\theta_{\mathbb{C}} \wedge \theta_{\mathbb{S}}$ by only taking into account events from SCCs (*strictly*) before e in the topological order of SSCs. If one of the effects is deferred (**delay** or **next**), the two triggers are evaluated at different time-points, and no event is shared. Any SAT result rejects the

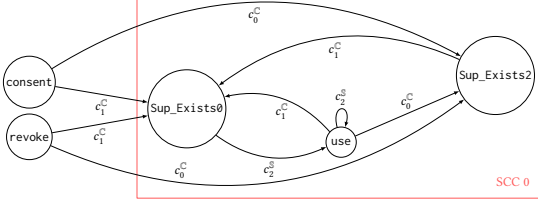


Fig. 6. EDG of the program in Figure 2. In its SCC, use is only suppressed (c_2^S) and Sup_Exists0/2 only caused ($c_{0,1}^C$), so the check succeeds.

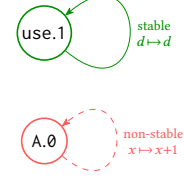


Fig. 7. Data-flow graph: the self-loop on use.1 (running example) is stable; the one on A.0 ($A(x) \Rightarrow A(x+1)$) is not.

Algorithm 4 Compilation algorithm

```

1: function COMPILE( $\mathcal{L}, \Gamma, R, \prec$ )
2:    $P \leftarrow$  declaration of all base/obligation events in  $R$ 
3:   for all  $p \in \Gamma$  do
4:     append to  $P$  the item realizing  $p$ 's body: table (past operator), agg, or filter let (non-guarded present) or let
5:   end for
6:   for all  $(\pi, \psi) \Rightarrow \varepsilon \in R$ , by SCC in order  $\prec$  do
7:     open a section fixpoint for the SCC if not already open
8:     append the rule  $[\pm] \hat{\varepsilon} := \text{trigger } \{ \pi \text{ if } \psi \}$ , where  $\hat{\varepsilon}$  carries a [delay  $n$ ] (resp. [next  $n$ ]) annotation if  $\varepsilon$  is
9:      $\diamond_{[n,n]} p(\dots)$  (resp.  $\circ \circ p(\dots)$  with  $n \circ$ )
10:   end for
11: return  $P$ 
12: end function

```

clause set. This rules out C_1 above but allows, e.g., $\{(\top, B(0)) \Rightarrow A(0), (\top, \neg B(0)) \Rightarrow \neg A(0)\}$, which is equivalent to $\Box A(0) \leftrightarrow B(0)$.

Termination. Termination is decided on a distinct *Data-Flow Graph* (DFG) whose nodes are event arguments $e.i$ and that contains an edge $e.i \rightarrow e'.j$ for any clause whose trigger mentions $e(\dots, t_i = x, \dots)$ and whose effect is $e'(\dots, t'_j, \dots)$ with x occurring in t'_j , and for any let $e'(\bar{x}) := \varphi$ such that φ mentions $e(\dots, t_i = x, \dots)$ and $x = x_j$ or, if φ is an aggregation, x_j is one of its results. An edge is *stable* iff none of its endpoints is the result of an aggregation and all function symbols in t'_j (if any) are stable under a fixed preorder $\preceq$ on $\mathbb{D}$, i.e. $\hat{f}(\bar{a}) \preceq a_k$ for some k and all $\bar{a}$ and f . Applying stable functions repeatedly to finitely many values yields finitely many values [28]. Enforcement terminates if no non-stable edge lies on a cycle. As a result, causing a fresh value on every iteration (e.g. $(A(x), \top) \Rightarrow A(x+1)$) is rejected, while, e.g., $(\text{use}(d), \top) \Rightarrow \neg \text{use}(d)$ is accepted because no new domain values are generated by that clause (Figure 7).

4.6 Compilation

Consider a closed formula $\Box \varphi$ with let-normal form $(\mathcal{L}, \varphi^b)$, the environment Γ and the guards θ computed for each let by TYPELET (Section 4.4), and a candidate clause set R that fulfills both checks in Section 4.5. Fix a topological order $\prec$ on the condensed SCCs of its EDG (sources first), which also induces an order on clauses. The compilation algorithm in Algorithm 4 turns $(\mathcal{L}, \Gamma, R)$ into an EF program that enforces $\Box \varphi$.

Each clause becomes an EF rule; a caused effect yields a $+$ rule, a suppressed effect a $-$ rule, and a deferred effect a proactive **delay/next** causation. Each let $p(\bar{x}) := \psi$ of $\mathcal{L}$ becomes an item built from the guards θ of TYPELET: $\diamond_{[a,b]} \varphi$ becomes **table** $p(\bar{x})$ **[window a b]** **:= add** $\{\theta\}$, $\bullet_{[a,b]} \varphi$ the

same **lagged** table, $\varphi_l S_{[a,b]} \varphi_r$ becomes **table** $p(\bar{x})$ **[window a b]** $:=$ **add** $\{\theta_l\}$ **remove** $\{\theta_r\}$, an aggregation an **agg let** over $\{\theta\}$, and a present body **let** $p(\bar{x}) := \{\theta\}$, or **filter let** $p(\bar{x}) := \{\text{if } \chi\}$ if p is filter-only. A trigger (π, ψ) becomes π **if** ψ , or **if** ψ if $\pi = \{\top\}$; compilation fails if a trigger or filter has no EF counterpart. The SCC order $\prec$ provides the EF sections.

Example 4.3. Running ENFLASH on φ_{law} (Example 2.3) yields the program of Figure 2. Let-normalization binds the since subformula $\neg\text{revoke}(u, c) S \text{consent}(u, c)$ to a let, which TYPELET compiles to the windowless table Since1: its right operand consent populates the **add** clause and its left operand revoke the **remove** clause. The whole quantifier block is kept together, so the negated body $\exists c, d, u. \text{use}(c, d, u) \wedge \neg\text{Since1}(c, u)$ is bound to the Boolean let Exists2. Suppressing it yields three clauses in one **fixpoint** section: $+\text{Sup_Exists0}/2$ raises the obligation event whenever a violation is present, and $-\text{use}$ then suppresses each offending $\text{use}(c, d, u)$ for which consent is missing or has been revoked. The resulting EDG (Figure 6) has a three-node SCC whose only suppressed event is use, so the conflict check passes; the only data-flow cycle is the stable self-loop $\text{use}(c, d, u) \Rightarrow \neg\text{use}(c, d, u)$ (Figure 7, left), so termination holds.

THEOREM 4.4 (COMPILATION CORRECTNESS). *Let $R \in \text{GENERATE}(\Box \varphi)$ be a candidate clause set passing the two checks of Section 4.5 and $P = \text{COMPILE}(\mathcal{L}, \Gamma, R, \prec)$. Then P is a sound enforcer for $\Box \varphi$.* [LBN Theorem_4_3]

5 Evaluation

We implement the compilation of Section 4 and the EF interpreter of Section 3 in a tool called ENFLASH. The compiler (7k LOC of OCaml) parses an MFOTL specification, runs let-normalization, guard extraction, enforcement rewriting, and the dependency analysis, discharges the cause/suppress checks with Z3, and emits an EF program. The interpreter (4k LOC of Rust) loads the EF program, computes a bound on its complexity as in Section 3, and exposes the reactive and proactive entry points μ and ν . We use Rust's pyo3 Python bindings to evaluate user-defined functions. ENFLASH is open-source and available [online](#). Our evaluation answers four research questions:

- RQ1** (Expressiveness) Does ENFLASH support the specifications of existing benchmarks and real applications?
- RQ2** (Latency) How does ENFLASH's latency compare to state-of-the-art first-order enforcers?
- RQ3** (Overhead in web applications) Is the end-to-end enforcement overhead small enough for user-facing operations in web applications?
- RQ4** (Comparison with permission engines) How does ENFLASH compare to a state-of-the-art non-temporal permission engine (Cedar) and its temporal extension (Dogwood) on access control, accounting for the cost of maintaining temporal state?

5.1 Experimental setup

All experiments run under Ubuntu 24.04.3 on an Intel Core Ultra 7 165U with 12 cores and 32GB of RAM. We run two types of experiments: tool-only benchmarks, where a trace from an existing benchmark is replayed to enforcement tools; and application benchmarks, where those tools serve as Policy Decision Points (PDP) for web applications [25].

In tool-only benchmarks, latency is measured as wall-clock time for the processing of single time points. We report per-time-point average and maximum latency for each run.

In application benchmarks, we measure full round-trip page latency using Python's requests library against a local server with the 'performance' CPU governor activated.

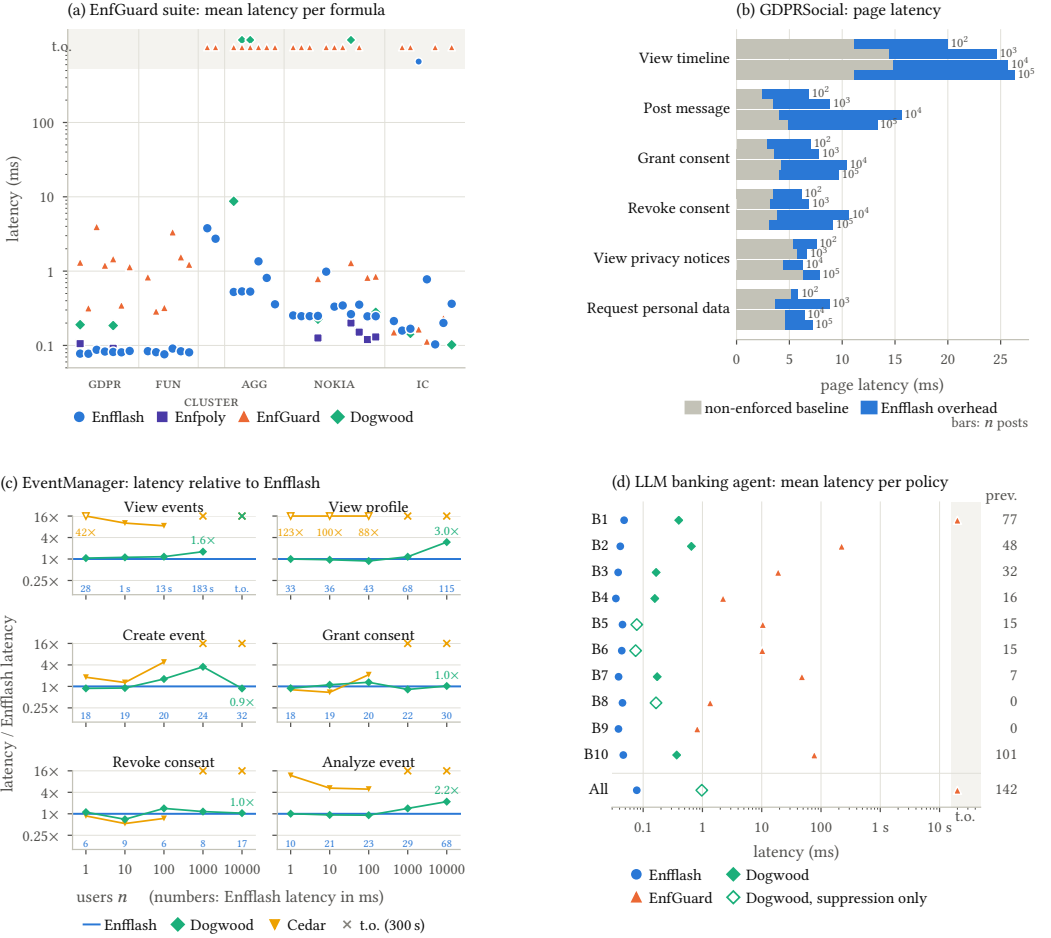


Fig. 8. Evaluation results (Tables 2–5); in (d), prev. is the number of attacks prevented out of 143.

5.2 Tool-only: the EnfGuard suite (RQ1,2)

We first evaluate ENFFLASH on EnfGuard’s six enforcement benchmarks, which span data protection (GDPR, FUN), aggregation-heavy policies (AGG), fault clustering (CLUSTER), infrastructure monitoring (IC), and database updates in a large system (NOKIA). We use all formulae in the benchmark with the largest available log. For each, we compile the specification and replay [33] the provided traces in real time, measuring latency against ENFFLASH, EnfGuard [28], Enfpoly [26], Monpoly [6], and Dogwood [9] on identical inputs, with 3 repetitions. We select EnfGuard and Enfpoly as the two fastest available MFOTL enforcers. Monpoly is an optimized MFOTL *monitor*, i.e. a tool that identifies but does not correct policy violations. Although Monpoly is therefore not a direct competitor to enforcement tools, it provides a challenging state-of-the-art latency baseline for a slightly simpler problem. Dogwood is a recent extension of Cedar with past-time temporal conditions over an event history. It supports suppression but no causation, and its temporal conditions are past-only, with bounded windows, no disjunction, and aggregations limited to counts and sums of integers.

Results (Figure 8a). Dogwood supports only 11 of the 40 formulae: other formulae require causing events (the deletion formulae and `CLUSTER`), future operators (`NOKIA`'s `del` and `ins` formulae), user-defined functions (`FUN`), or aggregations other than sums and counts (`AGG`'s `p4`–`p6`). On these, Dogwood is within 12% of `ENFFLASH` on 4, 2.4–17 $\times$ slower on 3, 3.6 $\times$ faster on 1, and times out on 3. `ENFFLASH` supports all policies in the benchmarks, with only the `NOKIA` formulae require rewriting to avoid nesting past and future operators. It processes 39 of the 40 within the timeout while `EnfGuard` processes 21. On 17 of the 20 processed by both, `ENFFLASH`'s average latency is 3–44 $\times$ lower than `EnfGuard`'s, which is faster on 2 `IC` formulae (1.4 $\times$, 7 $\times$). `Enfpoly`, which supports only 7 formulae, is 1.3–2.3 $\times$ faster than `ENFFLASH` on 5 `NOKIA` formulae. `ENFFLASH` is within 4 $\times$ of `Monpoly` on the 27 formulae processed by both (1.1–1.8 $\times$ faster on `GDPR`, up to 3.8 $\times$ slower on `AGG` and `NOKIA`); `Monpoly` times out on 4 `IC` formulae. Average (resp. max) latency is 0.08–3.8 ms (resp. 1–98 ms), below 1 ms on 36 formulae.

5.3 Application: GDPR-compliant social network (RQ1,3)

Our first deployment instruments `GDPRSOCIAL`, a microblogging app implemented in ca. 2.5k LOC of Python/Django that runs against an extensive GDPR specification encoded as a 400-line MFOTL formula. The GDPR specification is intrinsically *temporal*, with both statefulness (consent and revocation thereof must be persisted) and proactivity (personal data must be accessed, rectified, or deleted in a timely manner when users require it). `ENFFLASH` acts as an enforcement backend invoked through the `Instrlib` [25] library. We measure the latency of a set of representative application views: accessing a user's timeline, posting a message, giving or revoking consent for personalized advertisement, viewing privacy notices, and requesting access to one's data. Measurements are performed for $n \in \{10^2, 10^3, 10^4, 10^5\}$ posts by $n/100$ users over 10 repetitions after 5 warm-up calls. As a baseline, we perform measurements on a copy of the app stripped of all enforcement. Note that due to the temporal nature of the policy and the much higher latency of `EnfGuard`, no systematic experiments could be performed with competing tools. Dogwood cannot express the specification, as it requires causing events (e.g., deleting or providing personal data upon request).

Results (Figure 8b). `ENFFLASH` supports the large input formula. All views load in 6–27 ms, with enforcement inducing an overhead of 1–15 ms with respect to the non-enforced baseline. This loading time would typically be perceived as instantaneous by users [40, p. 135]. In particular, runtime latency does not grow with n , and hence enforcement scales well with database size.

5.4 Application: event-management platform (RQ1,3,4)

Our second deployment is `EVENTMANAGER`, an event-management platform whose access-control policy mixes role-based rules (e.g., only admins may add moderators) with stateful constraints (e.g., only event owners may promote an attendant of an event to manager of the event). This application was proposed by Basin et al. [5] and implemented in both `Cedar` [14] and `vActionGUI` [32] AC mechanisms. Because most of this policy is expressible without temporal operators when the roles are fixed, it admits a direct encoding in these two platforms, giving us two non-temporal baselines. However, the fact that these roles can change over time means that an extensive dynamic context must be passed to the non-temporal evaluator at every request, potentially increasing the latency. We straightforwardly translated the platform's access control policy into ca. 150 lines of MFOTL by replacing all the predicates of the form `hasRole( $\bar{x}$ )` by the $\neg \text{leftRole}(\bar{x}) \text{ S becomeRole}(\bar{x})$ formula. As a temporal baseline, we translated the MFOTL policy into Dogwood, which, unlike `Cedar`, also expresses the policy's consent rules: each $\neg \text{leftRole}(\bar{x}) \text{ S becomeRole}(\bar{x})$ becomes a `since` condition, and each disjunction a separate `permit`. The Dogwood version of the application is otherwise

identical to the ENFLASH one, with the same enforcement points and the same result cache. Measurements are performed for $n \in \{1, 10, \dots, 10^4\}$ users and $10n$ events; requests time out after 300 s (Cedar beyond $n = 100$, ‘view events’ beyond $n = 1000$).

Results (Figure 8c). Our ENFLASH implementation is faster than the Cedar baseline on 4 of 6 views and within 7 ms on the 2 consent views (where Cedar is up to $1.9\times$ faster), with $8\text{--}42\times$ and $88\text{--}123\times$ speedups on the ‘view events’ and ‘view profile’ pages, which query permissions about many entities. Dogwood is on par with ENFLASH up to $n = 100$, but its latency grows faster with the history: for $n \geq 1000$, it is $1.6\text{--}3.5\times$ slower on 4 views (e.g., $3.0\times$ on ‘view profile’ at $n = 10^4$).

5.5 Tool-only: LLM banking agent (RQ1,2)

Our third deployment enforces security policies on the tool calls of an LLM agent. We use the ‘banking’ suite of AgentDojo [16], an established benchmark for agentic security. In this suite, an agent operates a bank account using 11 tools (e.g., sending money, changing a password, reading files). The suite contains 16 benign and 144 attacked ground-truth runs, which we replay after translating all tool calls, tool outputs, and user prompts into log events. Each call is submitted to ENFLASH for approval before it executes, and suppressed calls are not executed.

We specify ten policies B1–B10 in MFOTL. For instance, B1 restricts payees to accounts mentioned by the user, paid before, or occurring in a file named by the user; B2 forbids payment subjects containing account data not supplied by the user; and B3 caps the amount sent to a payee within 24 hours at \$10,000. All policies are first-order; they also use unbounded past operators (B1,2,7,10), aggregations over metric windows (B3,4), and causation (B5,6,8,9). In contrast, existing runtime guardrails for LLM agents check either single tool calls [51] or propositional temporal properties [53]. Dogwood cannot cause events: we translate B1–B8 and B10 into Dogwood (for B5, B6, and B8, only their suppression part), emulating unbounded past operators by 100-year windows as in Section 5.2. We record the event log of each policy and replay it through ENFLASH, EnfGuard, and Dogwood one time point at a time, measuring latency as in Section 5.2 (timeout 600 s); on every log, the three tools suppress the same tool calls.

Results (Figure 8d). ENFLASH supports all ten policies and their conjunction. None of the policies suppresses a call in a benign run. Without enforcement, 143 of the 144 attacks succeed; the conjunction prevents 142 of them, with B1 alone preventing 77 attacks. Average latency is $0.03\text{--}0.05$ ms per time point for individual policies, and 0.08 ms for the conjunction of B1–B10. Dogwood can fully enforce 6 policies and partially enforce the suppression part of 3 additional ones (hollow markers in Figure 8d). An additional experiment on a large log of 23,320 shows that latency grows sublinearly with history size (52 to $64\ \mu\text{s}$ from the first to the last fifth of the run). EnfGuard is $20\text{--}5,570\times$ slower, while Dogwood is $1.8\text{--}16\times$ slower and $12\times$ slower on the conjunction.

5.6 Summary

ENFLASH supports all policies in the EnfGuard suite and the three application benchmarks, showing that the supported fragment of MFOTL is expressive enough to support a broad range of use cases (RQ1). Our tool achieves average per-event latency of $0.08\text{--}3.8$ ms, $3\text{--}44\times$ lower than EnfGuard on 17 of the 20 formulae on which EnfGuard does not time out, and within $4\times$ of Monpoly, a monitor solving a substantially simpler problem (RQ2). On our LLM banking agent benchmark, ENFLASH enforces ten security policies with $0.03\text{--}0.08$ ms average latency, $20\text{--}5,570\times$ faster than EnfGuard and $1.8\text{--}16\times$ faster than Dogwood, and prevents 142 of 143 successful prompt-injection attacks without suppressing any benign tool call (RQ1,2). In a real agentic deployment, ENFLASH’s sub-millisecond latency would be dominated by the costs of inference. In the GDPRSOCIAL application, enforcement adds $1\text{--}15$ ms of overhead despite the policy’s complexity (400 LOC), and this overhead

Table 1. Temporal enforcers (✓: supported, ~: partially, -: not supported)

	First-order data	Metric time	Aggregations	Suppression	Causation	Proactive deadlines	Sub-ms latency	Mechanized proofs
<i>Non-agentic</i>								
Timed enforcers [18, 19, 42, 46]	-	✓	-	✓	~	~	-	-
Hallé and Villemaire [21]	✓	-	-	✓	~	-	-	-
Enfpoly [26]	✓	✓	-	✓	✓	-	✓	-
Aceto et al. [1]	✓	-	-	✓	✓	-	-	-
Basin et al. [4]	-	✓	-	✓	✓	✓	-	-
WhyEnf [27]	✓	✓	-	✓	-	✓	-	-
Cedar [14]	✓	-	-	✓	-	-	✓	✓
EnfGuard [28]	✓	✓	✓	✓	✓	✓	-	-
<i>Agentic</i>								
AgentSpec [51]	~	-	-	✓	~	-	✓	-
Agent-C [30]	✓	-	-	✓	-	-	-	-
Dogwood [3, 9]	✓	✓	~	✓	-	-	~	-
FORGE [41]	✓	-	-	✓	-	-	-	-
C-Trace [29]	✓	-	-	✓	-	-	-	-
VIGIL [34]	-	-	-	✓	-	-	-	-
ContrAgent [53]	~	~	~	✓	-	-	✓	-
ENFLASH	✓	✓	✓	✓	✓	✓	✓	✓

does not significantly change with database size, keeping all views near-instantaneous (RQ3). ENFLASH matches or significantly improves on Cedar’s performance in our EVENTMANAGER application, up to 3.5× faster than Dogwood on long histories and up to 123× faster than Cedar’s full reload-and-evaluate pipeline in dynamic contexts (RQ4).

6 Related work

Runtime enforcement was introduced by Schneider in his seminal work on security automata [48]. Enforcement initially focused on execution cutting, i.e., terminating a system to prevent violations. Suppression [8, 20] and edit automata [35] were later introduced to allow for suppression and causation of events, respectively. However, these enforcers were still allowed to (unrealistically) buffer events before making decisions. This capability was later dropped [38], and other capabilities, such as delaying events [17, 43] and code inspection [44], were considered. In parallel, fine-grained enforcer capabilities were introduced, distinguishing between causable or suppressable and only-observable events [7]. Time-aware enforcers rely on timed automata as both a policy language and mechanism [18, 19, 42, 46] or as the translation target for logics such as MITL [36]. The first proactive enforcement algorithm was developed for DCR graphs [4], a propositional formalism [23].

First-order temporal enforcers exist for LTL-FO⁺, a first-order variant of future-only LTL [21], first-order Hennessy-Milner Logic with recursion (non-metric, suppression-only) [1, 2], as well as for several fragments of MFOTL, both past-only [26] and with past and future [27, 28].

Runtime enforcement as presented here is generic and thus provides theoretical underpinning for tools in many specialized domains, like access and usage control [14, 32], Android permission management [47], IoT [10], and LLM agent security [51], modulo appropriate instrumentation [25].

Recent years have seen intense research activity around runtime guardrails for LLM agents. Most enforcers in this space inspect each call in isolation: AgentSpec [51] evaluates trigger–predicate–action rules; Progent [49] and Conseca [50] restrict tool names and arguments to those needed by the user’s task; and CaMeL [15] and FIDES [13] track the provenance of values through capabilities and information-flow labels. LLM-based guards [11, 52] give no deterministic guarantees. Few systems support temporal policies: Agent-C [30] and VIGIL [34] check ordering constraints on the trace with an SMT solver, FORGE [41] evaluates Datalog policies, ContrAgent [53] enforces assume-guarantee contracts in propositional LTL_f with counters, and C-Trace [29] checks GDPR requirements expressed in MFOTL on agent traces. Amazon’s Bedrock AgentCore supports history-dependent policies written in Dogwood [3, 9]. None of these systems enforces real-time constraints over data-carrying events by both suppressing and causing events, e.g., limiting the amount sent to each payee within 24 hours while notifying the account owner of every sensitive change, as in our banking case study (Section 5.5). Table 1 summarizes these differences.

7 Conclusion

We presented ENFLASH, a novel first-order enforcer that is one to two orders of magnitude faster than the previous state-of-the-art, with comparable expressivity. ENFLASH is also up to an order of magnitude faster than the recently proposed Dogwood tool, developed for agentic security tasks; moreover, it is substantially more expressive and scales better with large logs. Even when deployed with very large policies, such as a 400-line MFOTL formula in a web application, ENFLASH induces only 1–15 ms of overhead. These results support the thesis that first-order runtime enforcement tools have become a mature alternative to ad-hoc frameworks for software security, including in the rapidly evolving field of agent security.

References

- [1] Luca Aceto, Ian Cassar, Adrian Francalanza, and Anna Ingólfssdóttir. 2023. Bidirectional Runtime Enforcement of First-Order Branching-Time Properties. *Logical Methods in Computer Science* 19 (2023).
- [2] Luca Aceto, Ian Cassar, Adrian Francalanza, and Anna Ingólfssdóttir. 2023. On first-order runtime enforcement of branching-time properties. *Acta Informatica* (2023), 1–67.
- [3] Amazon Web Services. 2026. Temporal policies – Amazon Bedrock AgentCore. <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-temporal.html>
- [4] David Basin, Søren Debois, and Thomas Hildebrandt. 2024. Proactive enforcement of provisions and obligations. *Journal of Computer Security* 32, 3 (2024), 247–289.
- [5] David Basin, Juan Guarnizo, Srdan Krstić, Hoàng Nguyen, and Martín Ochoa. 2023. Is Modeling Access Control Worth It?. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM, 2830–2844. doi:10.1145/3576915.3623196
- [6] David Basin, Matúš Harvan, Felix Klaedtke, and Eugen Zălinescu. 2011. MONPOLY: Monitoring usage-control policies. In *2nd International Conference on Runtime Verification (RV) (LNCS, Vol. 7186)*. Springer, 360–364.
- [7] David Basin, Vincent Jugé, Felix Klaedtke, and Eugen Zălinescu. 2013. Enforceable security policies revisited. *ACM Trans. Inf. Syst. Secur. (TISSEC)* 16, 1 (2013), 1–26.
- [8] Lujo Bauer, Jarred Ligatti, and David Walker. 2002. More enforceable security policies. In *Workshop on Foundations of Computer Security (FCS)*.
- [9] Marc Brooker, Joseph Tassarotti, and Jean-Baptiste Tristan. [n. d.]. Introducing Dogwood: runtime verification for AI agents. <https://aws.amazon.com/blogs/opensource/introducing-dogwood-runtime-verification-for-ai-agents/>
- [10] Z. Berkay Celik, Gang Tan, and Patrick D. McDaniel. 2019. IoTGuard: Dynamic Enforcement of Security and Safety Policy in Commodity IoT. In *26th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society.
- [11] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. Shieldagent: Shielding agents via verifiable safety policy reasoning. arXiv:2503.22738
- [12] Jan Chomicki. 1995. Efficient checking of temporal integrity constraints using bounded history encoding. *ACM Transactions on Database Systems (TODS)* 20, 2 (1995), 149–186.
- [13] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643
- [14] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John H. Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew Wells. 2024. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 670–697.
- [15] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating prompt injections by design. In *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 587–618.
- [16] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.
- [17] Yliès Falcone, Thierry Jéron, Hervé Marchand, and Srinivas Pinisetty. 2016. Runtime enforcement of regular timed properties by suppressing and delaying events. *Science of Computer Programming* 123 (2016), 2–41.
- [18] Yliès Falcone, Srdan Krstić, Giles Reger, and Dmitriy Traytel. 2021. A taxonomy for classifying runtime verification tools. *Int. J. Softw. Tools Technol. Transf.* 23, 2 (2021), 255–284.
- [19] Yliès Falcone and Srinivas Pinisetty. 2019. On the Runtime Enforcement of Timed Properties. In *19th International Conference on Runtime Verification, (RV) (LNCS, Vol. 11757)*, Bernd Finkbeiner and Leonardo Mariani (Eds.). Springer,

- 48–69.
- [20] Matthew Fredrikson, Richard Joiner, Somesh Jha, Thomas W. Reps, Phillip A. Porras, Hassen Saïdi, and Vinod Yegneswaran. 2012. Efficient Runtime Policy Enforcement Using Counterexample-Guided Abstraction Refinement. In *24th International Conference on Computer Aided Verification (CAV) (LNCS, Vol. 7358)*, P. Madhusudan and Sanjit A. Seshia (Eds.). Springer, 548–563.
 - [21] Sylvain Hallé and Roger Villemaire. 2012. Runtime Enforcement of Web Service Message Contracts with Data. *IEEE Trans. Serv. Comput.* 5, 2 (2012), 192–206.
 - [22] Klaus Havelund, Moran Omer, and Doron Peled. 2024. Operational and declarative runtime verification (keynote). In *7th ACM International Workshop on Verification and Monitoring at Runtime Execution (VORTEX)*, Davide Ancona and Giorgio Audrito (Eds.). ACM, 3–12.
 - [23] Thomas Hildebrandt, Raghava Rao Mukkamala, Tijs Slaats, and Francesco Zanitti. 2013. Contracts for cross-organizational workflows as timed dynamic condition response graphs. *The Journal of Logic and Algebraic Programming* 82, 5-7 (2013), 164–185.
 - [24] Manuel Hilty, David Basin, and Alexander Pretschner. 2005. On Obligations. In *10th European Symposium on Research in Computer Security (ESORICS) (LNCS, Vol. 3679)*, Sabrina De Capitani di Vimercati, Paul F. Syverson, and Dieter Gollmann (Eds.). Springer, 98–117. doi:10.1007/11555827_7
 - [25] François Hublet, David Basin, Linda Hu, Srđan Krstić, and Lennard Reese. 2025. Instrumenting runtime enforcement. In *25th International Conference on Runtime Verification (RV) (LNCS, Vol. 16087)*. Springer, 160–180. doi:10.1007/978-3-032-05435-7_10
 - [26] François Hublet, David Basin, and Srđan Krstić. 2022. Real-time policy enforcement with metric first-order temporal logic. In *27th European Symposium on Research in Computer Security (ESORICS) (LNCS)*, Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng (Eds.). Springer, 211–232.
 - [27] François Hublet, Leonardo Lima, David Basin, Srđan Krstić, and Dmitriy Traytel. 2024. Proactive real-time first-order enforcement. In *36th International Conference on Computer Aided Verification (CAV) (LNCS, Vol. 14682)*, Arie Gurfinkel and Vijay Ganesh (Eds.). Springer, 156–181.
 - [28] François Hublet, Leonardo Lima, David Basin, Srđan Krstić, and Dmitriy Traytel. 2025. Scaling up proactive enforcement. In *37th International Conference on Computer Aided Verification (CAV) (LNCS, Vol. 15933)*, Ruzica Piskac and Zvonimir Rakamaric (Eds.). Springer, 370–392.
 - [29] Nafiseh Kahani, Masoud Barati, and Diana Addae. 2026. Runtime Compliance Verification for AI Agents. arXiv:2606.19242
 - [30] Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. 2025. Enforcing Temporal Constraints for LLM Agents. arXiv:2512.23738
 - [31] Ron Koymans. 1990. Specifying real-time properties with metric temporal logic. *Real-time systems* 2, 4 (1990), 255–299.
 - [32] Srđan Krstić, Hoang Nguyen, and David Basin. 2024. Model-driven Privacy. *Proc. Priv. Enhancing Technol.* 2024, 1 (2024), 314–329. doi:10.56553/POPETS-2024-0018
 - [33] Srđan Krstić and Joshua Schneider. 2020. A benchmark generator for online first-order monitoring. In *20th International Conference on Runtime Verification (RV)*. Springer, 482–494.
 - [34] Ying Li, Yanju Chen, Hongbo Wen, Bosi Zhang, Hanzhi Liu, Peiran Wang, Yu Feng, and Yuan Tian. 2026. VIGIL: Runtime Enforcement of Behavioral Specifications in AI Agent Skills. arXiv:2606.26524
 - [35] Jay Ligatti, Lujo Bauer, and David Walker. 2005. Edit automata: Enforcement mechanisms for run-time security policies. *International Journal of Information Security* 4 (2005), 2–16.
 - [36] Oded Maler, Dejan Ničković, and Amir Pnueli. 2006. From MITL to timed automata. In *International conference on formal modeling and analysis of timed systems (FORMATS) (LNCS, Vol. 4202)*, Eugene Asarin and Patricia Bouyer (Eds.). Springer, 274–289.
 - [37] Leonardo de Moura and Sebastian Ullrich. 2021. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*. Springer, 625–635.
 - [38] Minh Ngo, Fabio Massacci, Dimitar Milushev, and Frank Piessens. 2015. Runtime Enforcement of Security Policies on Black Box Reactive Programs. In *42nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, Sriram K. Rajamani and David Walker (Eds.). ACM, 43–54.
 - [39] Hoàng Nguyễn, Srđan Krstić, and David Basin. 2026. Models Trump Code: An Empirical Analysis of GDPR Compliance. In *2026 ACM SIGSAC Conference on Computer and Communications Security*.
 - [40] Jakob Nielsen. 1993. *Usability engineering*. AP Professional.
 - [41] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, and Somesh Jha. 2026. Formal Policy Enforcement for Real-World Agentic Systems. arXiv:2602.16708
 - [42] Srinivas Pinisetty, Yliès Falcone, Thierry Jéron, and Hervé Marchand. 2015. TiPEX: A tool chain for timed property enforcement during execution. In *International Conference on Runtime Verification (RV)*. Springer, 306–320.

- [43] Srinivas Pinisetty, Ylies Falcone, Thierry Jéron, Hervé Marchand, Antoine Rollet, and Omer Nguena Timo. 2014. Runtime enforcement of timed properties revisited. *Formal Methods Syst. Des.* 45 (2014), 381–422.
- [44] Srinivas Pinisetty, Viorel Preoteasa, Stavros Tripakis, Thierry Jéron, Ylies Falcone, and Hervé Marchand. 2017. Predictive runtime enforcement. *Formal Methods Syst. Des.* 51, 1 (2017), 154–199.
- [45] Amir Pnueli. 1977. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 46–57.
- [46] Matthieu Renard, Antoine Rollet, and Ylies Falcone. 2017. GREP: Games for the Runtime Enforcement of Properties. In *International Conference on Testing Software and Systems (ICTSS) (LNCS, Vol. 10533)*, Nina Yevtushenko, Ana Rosa Cavalli, and Hüsni Yenigün (Eds.). Springer, 259–275.
- [47] Alireza Sadeghi, Reyhaneh Jabbarvand, Negar Ghorbani, Hamid Bagheri, and Sam Malek. 2018. A temporal permission analysis and enforcement framework for Android. In *40th International Conference on Software Engineering, (ICSE)*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 846–857. doi:10.1145/3180155.3180172
- [48] Fred B. Schneider. 2000. Enforceable security policies. *ACM Trans. Inf. Syst. Secur. (TISSEC)* 3, 1 (2000), 30–50.
- [49] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703
- [50] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. In *Workshop on Hot Topics in Operating Systems (HotOS)*.
- [51] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. 2938–2950.
- [52] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, et al. 2024. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. arXiv:2406.09187
- [53] Yifeng Xiao and Pierluigi Nuzzo. 2026. Symbolic Temporal Supervision of LLM Agents Using Contracts. arXiv:2609.18128

A Detailed evaluation results

Tables 2–5 give the results summarized in Figure 8.

B A type system for the enforceable fragment

The fragment enforced by ENFFLASH is defined implicitly by the compilation of Section 4: a formula is supported if let-normalization, TYPELET, and the enforcement rewriting succeed. We now characterize this fragment by a type system over formulae in let-normal form, which we call EF-MFOTL, and show that typability coincides with successful compilation.

Contexts. A context Γ assigns to each let-bound event p a set of capabilities $\Gamma(p) \subseteq \{\mathbb{G}, \mathbb{C}, \mathbb{S}\}$: p may be used as a guard ($\mathbb{G}$), caused ($\mathbb{C}$), or suppressed ($\mathbb{S}$). Base events are always enumerable, and their causability and suppressability are fixed by the signature. We write m_Γ for the set of enumerable predicates of Γ .

Guardedness. The judgment $\Gamma \vdash \varphi : \text{GRD}(x)^p$, for a polarity $p \in \{+, -\}$, states that the values of x satisfying $p\varphi$ can be enumerated from the enumerable predicates (Figure 9). For a set of variables X , $\Gamma \vdash \varphi : \mathbb{G}_X^p$ holds iff $\Gamma \vdash \varphi : \text{GRD}(x)^p$ for all $x \in X$. A trigger (π, ψ) guards x if every disjunct $\kappa \in \pi$ contains an atom with argument x , or $\Gamma \vdash \psi : \text{GRD}(x)^+$.

$$\begin{array}{llll}
 \text{PRED} \frac{p \in m_\Gamma \quad x \in \bar{i}}{\Gamma \vdash p(\bar{i}) : \text{GRD}(x)^+} & \text{EQ} \frac{}{\Gamma \vdash x = c : \text{GRD}(x)^+} & \text{VAC} \frac{}{\Gamma \vdash \top : \text{GRD}(x)^-} & \text{NEG} \frac{\Gamma \vdash \varphi : \text{GRD}(x)^{-p}}{\Gamma \vdash \neg\varphi : \text{GRD}(x)^p} \\
 \text{AND}_L^+ \frac{\Gamma \vdash \varphi : \text{GRD}(x)^+}{\Gamma \vdash \varphi \wedge \psi : \text{GRD}(x)^+} & \text{AND}_R^+ \frac{\Gamma \vdash \psi : \text{GRD}(x)^+}{\Gamma \vdash \varphi \wedge \psi : \text{GRD}(x)^+} & \text{AND}^- \frac{\Gamma \vdash \varphi : \text{GRD}(x)^- \quad \Gamma \vdash \psi : \text{GRD}(x)^-}{\Gamma \vdash \varphi \wedge \psi : \text{GRD}(x)^-}
 \end{array}$$

Fig. 9. Guardedness (c is a constant)

$$\begin{array}{c}
\text{Top}^{\mathbb{C}} \frac{}{\Gamma \vdash \top : \mathbb{C} \triangleright \emptyset} \quad \text{Ev}^{\mathbb{C}} \frac{e \in \mathbb{C}}{\Gamma \vdash e(\bar{i}) : \mathbb{C} \triangleright \{(\top, \top) \Rightarrow e(\bar{i})\}} \quad \text{Ev}^{\mathbb{S}} \frac{e \in \mathbb{S}}{\Gamma \vdash e(\bar{i}) : \mathbb{S} \triangleright \{(e(\bar{i}), \top) \Rightarrow \neg e(\bar{i})\}} \\
\\
\text{LET}^{\mathbb{C}} \frac{\mathbb{C} \in \Gamma(e)}{\Gamma \vdash e(\bar{i}) : \mathbb{C} \triangleright \{(\top, \top) \Rightarrow \text{Cau}_-e(\bar{i})\}} \quad \text{LET}^{\mathbb{S}} \frac{\mathbb{S} \in \Gamma(e)}{\Gamma \vdash e(\bar{i}) : \mathbb{S} \triangleright \{(e(\bar{i}), \top) \Rightarrow \text{Sup}_-e(\bar{i})\}} \\
\\
\text{NEG}^{\mathbb{C}} \frac{\Gamma \vdash \varphi : \mathbb{S} \triangleright \Delta}{\Gamma \vdash \neg \varphi : \mathbb{C} \triangleright \Delta} \quad \text{NEG}^{\mathbb{S}} \frac{\Gamma \vdash \varphi : \mathbb{C} \triangleright \Delta}{\Gamma \vdash \neg \varphi : \mathbb{S} \triangleright \Delta} \quad \text{AND}^{\mathbb{C}} \frac{\Gamma \vdash \varphi : \mathbb{C} \triangleright \Delta_1 \quad \Gamma \vdash \psi : \mathbb{C} \triangleright \Delta_2}{\Gamma \vdash \varphi \wedge \psi : \mathbb{C} \triangleright \Delta_1 \cup \Delta_2} \\
\\
\text{AND}^{\mathbb{S}} \frac{\Gamma \vdash \varphi_j : \mathbb{S} \triangleright \Delta \quad \varphi_i \text{ present for } i \neq j}{\Gamma \vdash \bigwedge_i \varphi_i : \mathbb{S} \triangleright \Delta^{\wedge_{i \neq j} \varphi_i}} \quad \text{EX}^{\mathbb{C}} \frac{\Gamma \vdash \varphi : \mathbb{C} \triangleright \Delta}{\Gamma \vdash \exists x. \varphi : \mathbb{C} \triangleright \Delta[0/x]} \\
\\
\text{EX}^{\mathbb{S}} \frac{\Gamma \vdash \varphi : \mathbb{S} \triangleright \Delta \quad \text{every trigger of } \Delta \text{ guards } x}{\Gamma \vdash \exists x. \varphi : \mathbb{S} \triangleright \Delta \downarrow_x} \\
\\
\text{FUT}_{\diamond}^{\mathbb{C}} \frac{\Gamma \vdash \varphi : \mathbb{C} \triangleright \Delta \quad \Delta \neq \emptyset \text{ unconditional} \quad a \leq b \quad 1 \leq b < \infty}{\Gamma \vdash \diamond_{[a,b]} \varphi : \mathbb{C} \triangleright \diamond_{[b,b]} \Delta} \\
\\
\text{FUT}_{\circ}^{\mathbb{C}} \frac{\Gamma \vdash \varphi : \mathbb{C} \triangleright \Delta \quad \Delta \neq \emptyset \text{ unconditional} \quad 1 \leq b}{\Gamma \vdash \circ_{[0,b]} \varphi : \mathbb{C} \triangleright \circ \Delta} \quad \text{FUT}_{\circ^n}^{\mathbb{C}} \frac{\Gamma \vdash \varphi : \mathbb{C} \triangleright \Delta \quad \Delta \text{ unconditional} \quad 1 \leq n}{\Gamma \vdash \circ^n \varphi : \mathbb{C} \triangleright \circ^n \Delta}
\end{array}$$

Fig. 10. Causation and suppression

LEMMA B.1. (1) $m_\Gamma \vdash (\pi, \psi) \rightsquigarrow_x^p (\pi', \psi')$ holds for some (π', ψ') iff every $\kappa \in \pi$ binds x or $\Gamma \vdash \psi : \text{GRD}(x)^p$. In particular, a guard for x can be extracted from a trigger iff the trigger guards x .
(2) $\text{Guards}_X^{m_\Gamma}(\Phi) \neq \perp$ iff $\Gamma \vdash \Phi : \mathbb{G}_X^+$.

[LEMMA_A_1]

Causation and suppression. The judgment $\Gamma \vdash \varphi : \alpha \triangleright \Delta$ with $\alpha \in \{\mathbb{C}, \mathbb{S}\}$ states that the clauses Δ cause (resp. suppress) φ (Figure 10). We write Δ^ψ for $\{(\pi, \varphi \wedge \psi) \Rightarrow \varepsilon \mid (\pi, \varphi) \Rightarrow \varepsilon \in \Delta, \Delta[c/x] \text{ for the substitution of a constant } c \text{ for } x \text{ in every clause (where defined; we use a canonical constant 0), and } \Delta_{\downarrow x} \text{ for } \{(\pi', \psi') \Rightarrow \varepsilon \mid (\pi, \psi) \Rightarrow \varepsilon \in \Delta, m_\Gamma \vdash (\pi, \psi) \rightsquigarrow_x^+ (\pi', \psi')\}$. A clause set is *unconditional* if all its clauses have the form $(\top, \top) \Rightarrow e(\bar{i})$; then $\diamond_{[b,b]} \Delta$ and $\circ^n \Delta$ defer all effects.

Let bindings. Let let $e_1(\bar{x}_1) = \varphi_1$ in $\dots$ let $e_k(\bar{x}_k) = \varphi_k$ in $\cap \chi_1 \wedge \dots \wedge \cap \chi_n$ be in let-normal form, and $\chi = \chi_1 \wedge \dots \wedge \chi_n$. For each let body φ_i , let $\hat{\varphi}_i$ be the formula producing its tuples (ψ for $\exists \bar{y}. \psi$, where ψ has no leading existential; φ_r for $\varphi_l S_l \varphi_r$; φ for $\bullet_I \varphi$ and for an aggregation $\bar{o} \leftarrow \omega(\bar{t}; \bar{g}) \varphi$, whose results $\bar{o}$ are then excluded from $\bar{x}_i$). We write $\Gamma_{<}$ for Γ restricted to $e_1, \dots, e_{i-1}$, and require that

- $\mathbb{G} \in \Gamma(e_i)$ iff $\Gamma_{<i} \vdash \hat{\phi}_i : \mathbb{G}_{\bar{x}_i \bar{y}}^+$ and, if $\varphi_i = \varphi_l \text{ SI } \varphi_r$, also $\Gamma_{<i} \vdash \varphi_l : \mathbb{G}_{\bar{x}_i}^-$ (the **remove** clause of the table), where $\bar{x}_i \bar{y}$ stands for $\text{fv}(\hat{\phi}_i)$ for an aggregation;
- if φ_i is temporal or an aggregation, then $\mathbb{G} \in \Gamma(e_i)$, and the terms $\bar{t}$ of an aggregation only use variables of $\hat{\phi}_i$;
- $\mathbb{C} \in \Gamma(e_i)$ (resp. $\mathbb{S} \in \Gamma(e_i)$) iff $\mathbb{G} \in \Gamma(e_i)$ and $\Gamma_{<i} \vdash \text{cau}(\varphi_i) : \mathbb{C} \triangleright \Delta_i$ (resp. $\Gamma_{<i} \vdash \text{sup}(\varphi_i) : \mathbb{S} \triangleright \Delta_i$) for some Δ_i ,

where $\text{cau}(\psi) = \sup(\psi) = \psi$ for present bodies, $\text{cau}(\varphi_l S_I \varphi_r) = \varphi_r$ if $0 \in I$ (and undefined otherwise), $\sup(\varphi_l S_I \varphi_r) = \varphi_l \vee \varphi_r$ if $0 \in I$ and φ_l otherwise, and neither is defined for $\bullet_I$ or aggregations. Lets without $\mathbb{G}$ are *filter-only*: they can be used in filters, but not as guards.

Definition B.2 (EF-MFOTL). A formula $\Box \varphi$ is in EF-MFOTL with clause set Δ if its let-normal form, with lets typed by some context Γ as above, satisfies $\Gamma \vdash \chi : \mathbb{C} \triangleright \Delta$.

THEOREM B.3. $\Box \varphi$ is in EF-MFOTL with clause set Δ iff the compilation of Section 4 succeeds on $\Box \varphi$ with candidate clause set Δ , i.e., TYPELET accepts all lets, and $\Gamma \vdash \chi \hookrightarrow^{\mathbb{C}} C$ for the resulting Γ and some $C \ni \Delta$. [LEAN Theorem_A_2]

PROOF SKETCH. The rules of Figure 10 mirror those of Figure 5, producing a single clause set instead of a set of alternatives: by induction on derivations, $\Gamma \vdash \varphi : \alpha \triangleright \Delta$ iff $\Gamma \vdash \varphi \hookrightarrow^{\alpha} C$ for some $C \ni \Delta$. The side condition of $\text{Ex}^{\mathbb{S}}$ is equivalent to that of the rewrite rule by Lemma B.1(1), and the enumerability conditions on let bodies are equivalent to the success of Guards by Lemma B.1(2). Given a typing, the guards and realizations are obtained from the typing derivations of the let bodies; conversely, capabilities are read off the guards and realizations computed by TYPELET. $\square$

Together with Theorem 4.4, every formula of EF-MFOTL whose clause set passes the checks of Section 4.5 is soundly enforced by the compiled EF program. Lemma B.1 and Theorem B.3 are formalized in Lean 4, together with the soundness of compilation and of the enforcement loop.

Example B.4. Consider φ_{del} from Example 2.3, with let-normal form $\Box \neg \exists d, u. \varphi_0$ for $\varphi_0 = \text{deletion_request}(d, u) \wedge \neg \Diamond_{[0,30]} \text{delete}(d, u)$ and no lets. By $\text{Ev}^{\mathbb{C}}$ and $\text{FUT}_{\Diamond}^{\mathbb{C}}$, $\Gamma \vdash \Diamond_{[0,30]} \text{delete}(d, u) : \mathbb{C} \triangleright \{(\top, \top) \Rightarrow \Diamond_{[30,30]} \text{delete}(d, u)\}$; by $\text{NEG}^{\mathbb{S}}$ and $\text{AND}^{\mathbb{S}}$, $\Gamma \vdash \varphi_0 : \mathbb{S} \triangleright \{(\top, \text{deletion_request}(d, u)) \Rightarrow \Diamond_{[30,30]} \text{delete}(d, u)\}$. The trigger guards u and d (rule PRED, as base events are enumerable), so two applications of $\text{Ex}^{\mathbb{S}}$ and $\text{NEG}^{\mathbb{C}}$ yield $\Gamma \vdash \neg \exists d, u. \varphi_0 : \mathbb{C} \triangleright \{(\text{deletion_request}(d, u), \top) \Rightarrow \Diamond_{[30,30]} \text{delete}(d, u)\}$.

Table 2. Latency in ms—*mean (max)*—on the largest logs on the EnfGuard benchmark suite at maximal real-time acceleration

Policy	Enforcement								Monitoring	
	EnfFlash	Enfpoly	EnfGuard	Dogwood					Monpoly	
<i>GDPR (timeout = 60 s)</i>										
consent	0.08 (1)	0.11 (1)	1.32 (6)	0.19 (1)					0.14 (2)	
deletion	0.08 (1)	–	0.32 (3)	–					0.09 (1)	
gdpr	0.09 (2)	–	3.97 (13)	–					0.11 (3)	
information	0.08 (1)	–	1.20 (7)	–					0.09 (2)	
lawfulness	0.08 (1)	0.09 (2)	1.46 (7)	0.19 (2)					0.10 (1)	
limitation	0.08 (1)	–	0.35 (59)	–					–	
sharing	0.08 (1)	–	1.14 (5)	–					0.10 (1)	
<i>FUN (timeout = 60 s)</i>										
consent	0.08 (2)	–	0.83 (5)	–					–	
consent_management	0.08 (1)	–	0.29 (2)	–					–	
deletion	0.08 (1)	–	0.32 (3)	–					–	
gdpr	0.09 (1)	–	3.36 (12)	–					–	
information	0.08 (2)	–	1.55 (9)	–					–	
sharing	0.08 (2)	–	1.23 (5)	–					–	
<i>CLUSTER (timeout = 60 s)</i>										
dbscan	3.78 (12)	–	t.o.	–					–	
grubbs	2.73 (7)	–	t.o.	–					–	
<i>AGG (timeout = 120 s)</i>										
p1	0.52 (4)	–	t.o.	8.71 (45)					0.19 (3)	
p2	0.53 (3)	–	t.o.	t.o.					0.32 (3)	
p3	0.53 (2)	–	t.o.	t.o.					0.39 (3)	
p4	1.35 (15)	–	t.o.	–					0.52 (3)	
p5	0.81 (3)	–	t.o.	–					0.21 (2)	
p6	0.36 (4)	–	t.o.	–					0.44 (3)	
<i>NOKIA (timeout = 120 s)</i>										
del-1-2	0.25 (3)	–	t.o.	–					0.20 (19)	
del-2-3	0.25 (12)	–	t.o.	–					0.12 (3)	
del-3-2	0.25 (32)	–	t.o.	–					0.13 (3)	
delete	0.25 (22)	0.13 (3)	0.79 (11)	0.23 (4)					0.13 (4)	
ins-1-2	0.98 (34)	–	t.o.	–					1.06 (112)	
ins-2-3	0.33 (3)	–	t.o.	–					0.41 (9)	
ins-3-2	0.35 (22)	–	t.o.	–					0.40 (7)	
insert	0.26 (14)	0.20 (3)	1.30 (25)	t.o.					0.20 (5)	
script1	0.35 (23)	0.15 (2)	t.o.	–					0.28 (3)	
select	0.25 (12)	0.12 (3)	0.82 (12)	0.25 (4)					0.13 (3)	
update	0.25 (13)	0.13 (4)	0.85 (15)	0.28 (5)					0.13 (4)	
<i>IC (timeout = 600 s)</i>										
block_validation_latency	0.21 (32)	–	0.15 (7)	–					t.o.	
clean_logs	0.16 (23)	–	t.o.	–					0.12 (5)	
finalization_consistency	0.17 (32)	–	t.o.	0.15 (7)					0.08 (4)	
finalized_height	t.o.	–	0.17 (5)	–					t.o.	
logging_behavior	0.78 (98)	–	0.11 (25)	–					t.o.	
reboot_count	0.10 (23)	–	t.o.	–					0.11 (4)	
replica_divergence	0.20 (32)	–	0.23 (5)	–					0.08 (4)	
unauthorized_connections	0.36 (95)	–	t.o.	0.10 (6)					t.o.	

Table 3. GDPRSocial page latency in ms (+overhead to non-enforced baseline) for representative application views

View	$n = 10^2$		$n = 10^3$		$n = 10^4$		$n = 10^5$	
View timeline	19.98	+9	24.63	+10	25.69	+11	26.32	+15
Post message	6.82	+4	8.84	+5	15.61	+12	13.31	+8
Grant consent	7.00	+4	7.80	+4	10.44	+6	9.68	+6
Revoke consent	6.14	+3	6.87	+4	10.63	+7	9.14	+6
View privacy notices	7.56	+2	6.68	+1	6.27	+2	7.84	+2
Request personal data	5.83	+1	8.79	+5	6.40	+2	7.19	+3

Table 4. EVENTMANAGER page latency in ms: ENFLASH (E) vs. Dogwood (D) vs. Cedar (C). t.o.: timeout (300 s per request)

View	$n = 1$			$n = 10$			$n = 100$			$n = 1000$			$n = 10000$		
	E	D	C	E	D	C	E	D	C	E	D	C	E	D	C
View events	27.79	29.44	1 s	1 s	1 s	11 s	13 s	15 s	112 s	183 s	294 s	t.o.	t.o.	t.o.	t.o.
View profile	33.23	33.15	4 s	36.26	34.60	3 s	43.29	38.06	3 s	68.04	78.70	t.o.	114.60	340.63	t.o.
Create event	17.97	15.83	32.48	19.23	17.32	24.73	20.26	32.65	97.20	24.19	85.68	t.o.	32.43	28.41	t.o.
Grant consent	18.17	16.15	14.72	19.12	21.23	13.04	19.55	25.50	41.53	22.38	18.40	t.o.	29.67	30.44	t.o.
Revoke consent	6.14	6.82	5.35	9.30	6.52	4.96	6.46	9.17	4.79	7.70	8.89	t.o.	17.23	17.98	t.o.
Analyze event	10.03	10.08	119.38	21.44	19.94	112.52	23.04	21.07	113.15	28.64	40.55	t.o.	68.29	149.32	t.o.

Table 5. LLM banking agent (AgentDojo): attacks prevented among those successful without enforcement (Prev.), benign runs with a suppressed call (FP), attacked runs whose user task is completed (Util.), and latency in ms—*mean (max)*—on the recorded logs. †: suppression part only

Policy	Prev.	FP	Util.	EnfFlash	EnfGuard	Dogwood
– No enforcement	0/143	0/16	126/144			
B1 Payee provenance	77/143	0/16	131/144	0.05 (0.3)	t.o.	0.40 (5)
B2 No leak in subject	48/143	0/16	134/144	0.04 (0.5)	222.79 (1278)	0.65 (20)
B3 Daily cap per payee	32/143	0/16	128/144	0.04 (0.2)	18.93 (100)	0.17 (0.8)
B4 Velocity	16/143	0/16	121/144	0.03 (0.1)	2.23 (9)	0.16 (0.7)
B5 Standing-order redirect	15/143	0/16	128/144	0.04 (0.2)	10.42 (50)	0.08† (1)
B6 Password change	15/143	0/16	128/144	0.04 (0.4)	10.22 (52)	0.07† (2)
B7 Cool-down after credential change	7/143	0/16	124/144	0.04 (0.1)	47.66 (345)	0.17 (2)
B8 Quarantine	0/143	0/16	126/144	0.04 (0.2)	1.35 (5)	0.16† (0.6)
B9 Audit	0/143	0/16	126/144	0.04 (0.1)	0.82 (4)	–
B10 Invoice consistency	101/143	0/16	131/144	0.05 (0.2)	77.04 (546)	0.37 (4)
– All (B1–B10)	142/143	0/16	55/144	0.08 (0.3)	t.o.	0.98† (28)